EABench: A Configurable End-to-End Platform for Evaluating Enterprise LLM Agents

HAOYI XIONG, Microsoft Corporation, China
YI LIU, Microsoft Corporation, China
NAN CHEN, Microsoft Corporation, China
BIN ZHU, Microsoft Corporation, China
ZHONGXIAN JIA, Microsoft Corporation, China
YIQIN YU, Microsoft Corporation, China
YI WANG, Microsoft Corporation, China

Enterprise deployment of LLM agents demands more than a static benchmark: researchers need a controllable environment that can generate realistic organizational data, instantiate alternative agent architectures, synthesize challenging evaluation cases, and expose failures in a form that is easy to debug. Existing agent benchmarks capture portions of this problem, but they typically fix the underlying dataset, hard-code the evaluation metrics, or omit the access-controlled and cross-modal structure that defines enterprise knowledge work. We present **EABench**, an end-to-end experimental platform built around three contributions. First, we formally characterize enterprise agent evaluation as a systems problem involving access-controlled, cross-modal, and temporally structured organizational data, and identify three recurring gaps in existing benchmarks—static datasets, hard-coded metrics, and absent debugging support. Second, we design and implement an end-to-end solution that covers scenario-aware synthetic tenant generation, a fully hot-swappable agent runtime, automated evaluation-dataset construction, a multi-dimensional LLM-as-a-judge harness, and an interactive debugging interface—all driven by declarative YAML files without code changes. Third, we conduct case studies and controlled experiments across three synthetic enterprise tenants (legal, education, and AI/software) and four agent configurations (three ReAct variants spanning two prompt styles and two model sizes, plus a plan-and-execute Researcher), providing a systematic analysis of how agent architecture, prompt engineering, model size, and tenant scenario jointly affect assertion satisfaction, citation grounding, and operational cost.

CCS Concepts: • **Computing methodologies** → **Artificial intelligence**; **Machine learning**.

Additional Key Words and Phrases: LLM agents, enterprise AI, benchmarking, evaluation frameworks, synthetic data generation

Authors' Contact Information: Haoyi Xiong, Microsoft Corporation, Beijing, China; Yi Liu, Microsoft Corporation, Beijing, China; Nan Chen, Microsoft Corporation, Beijing, China; Bin Zhu, Microsoft Corporation, Beijing, China; Zhongxian Jia, Microsoft Corporation, Beijing, China; Yiqin Yu, Microsoft Corporation, Beijing, China; Yi Wang, Microsoft Corporation, Beijing, China.

Manuscript submitted to ACM

ACM Reference Format:
Haoyi Xiong, Yi Liu, Nan Chen, Bin Zhu, Zhongxian Jia, Yiqin Yu, and Yi Wang. 2026. EABench: A Configurable End-to-End Platform for Evaluating Enterprise LLM Agents. *ACM Trans. Intell. Syst. Technol.* 1, 1, Article 1 (April 2026), 44 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Enterprise agents operate in an environment that is substantially harder than the public-web and API-centric settings emphasized by most existing benchmarks. A useful enterprise assistant must resolve people and permissions, search across heterogeneous artifacts, reconstruct event timelines from partial evidence, and synthesize answers that remain grounded in access-controlled sources. These demands are compounded by multi-hop reasoning requirements [20]: enterprise queries rarely have single-document answers but instead require chaining evidence across emails, meeting transcripts, files, and calendar events—precisely the kind of cross-document knowledge retrieval that retrieval-augmented generation (RAG) [9] has made feasible for open-domain question answering, but which enterprise deployments demand at organizational scale under strict permission boundaries. Together, these properties make enterprise evaluation fundamentally a systems problem: the benchmark must model data generation, retrieval infrastructure, execution policy, and assessment criteria together rather than as isolated components.

The urgency of this challenge mirrors the rapid pace of enterprise AI adoption. Commercial deployments such as Microsoft 365 Copilot [12] already process millions of knowledge-work queries daily, yet independent benchmarking is severely hampered by the absence of controlled environments that can reproduce access-controlled organizational data at scale. Even the most capable frontier models struggle in this regime: the OfficeQA Pro benchmark [14]—which requires grounded reasoning over a large, heterogeneous document corpus—found that frontier agents achieve under 12% accuracy when relying on web access alone and under 35% even when given direct corpus access, confirming that enterprise-grade evaluation remains far from solved. General-purpose benchmarks such as GAIA [11], OSWorld [19], and SWE-bench [8] have pushed the frontier of multi-step AI assistance in web navigation, desktop GUIs, and software repositories, but they abstract away the access-controlled, cross-modal, and temporally structured character of real enterprise knowledge work.

Alongside these general-purpose efforts, enterprise-specific benchmarks—WebArena [25], AgentBench [10], ToolBench [15], WorkArena [5], EnterpriseBench [17], DRBench [1], and HERB [3]—each capture important slices of agent capability, but they leave three recurring gaps for enterprise research. First, they generally rely on static datasets or fixed task pools, which prevents researchers from generating new organizational histories or stress-testing a method under controlled scenario variation; while dynamically generated benchmarks have been proposed [16], they address only narrow task domains and do not generalize to multi-source enterprise knowledge work. Second, they rarely expose the agent runtime and the judge as independently configurable objects, making systematic ablations across models, prompts, tools, embeddings, and scoring criteria unnecessarily expensive. Third, they under-specify the debugging problem: when an enterprise agent fails, researchers need to inspect the search trajectory, tool usage, citations, and permission context rather than only the final answer.

EABench addresses these gaps through three contributions.

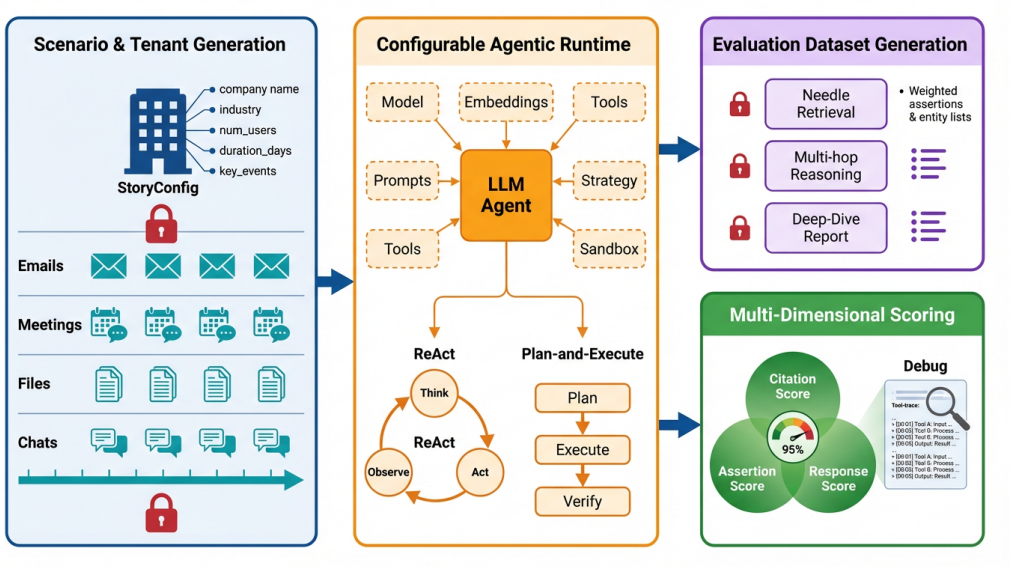

Fig. 1. EABench graphical overview. The four coupled platform stages proceed from synthetic tenant generation (Stage 1), through configurable agent execution (Stage 2) and automated evaluation-dataset construction (Stage 3), to multi-dimensional scoring with interactive debugging (Stage 4), forming a single reproducible experimental pipeline.

- **New problem formulation.** We characterize enterprise agent evaluation as a systems problem that combines access-controlled retrieval, cross-modal artifact graphs, temporal event reasoning, and multi-hop synthesis. Surveying existing benchmarks reveals three recurring gaps—static datasets, hard-coded metrics, and absent debugging support—that motivate a new experimental platform.
- **End-to-end configurable solution.** We design and implement EABench, an integrated pipeline covering (i) scenario-aware synthetic tenant generation with controllable company size, industry, and narrative event structure; (ii) a fully configurable agent runtime in which the reasoning model, embedding model, agent architecture, system prompt, tool set, and sandbox backend are all hot-swappable via declarative YAML files; (iii) automated evaluation-dataset construction that yields three query types (targeted artifact search, multi-hop reasoning, and comprehensive report) paired with user-level access context, ground-truth entity lists, and natural-language assertions; (iv) a multi-dimensional LLM-as-a-judge harness whose judge model and prompts are likewise YAML-configurable; and (v) an interactive Streamlit debugging interface that surfaces tool traces, retrieval evidence, and citations over the same runtime used for batch evaluation.
- **Case studies and experiments.** We instantiate EABench across three synthetic enterprise tenants—*Bertrand & Co.* (legal), *University of Cambford* (education), and *ZAI Intelligence* (AI/software)—and evaluate four agent configurations (three ReAct variants that vary prompt verbosity and model size, plus a plan-and-execute RESEARCHER) under identical access constraints. Controlled ablations isolate the effects of agent architecture, prompt engineering, model size, tenant scenario, and query type on assertion satisfaction, citation grounding, and operational cost, and show that the cost–quality frontier

Table 1. Comparison of EABench with representative agent benchmarks. ✓ indicates direct support, Partial indicates limited or indirect support, and – indicates that the capability is absent from the published benchmark design.

Feature	WebArena	AgentBench	ToolBench	WorkArena	DRBench	HERB	EnterpriseBench	EABench
End-to-end evaluation pipeline	–	–	–	–	Partial	Partial	Partial	✓
Configurable data generation	–	–	–	–	Partial	✓	Partial	✓
Configurable agent runtime	–	–	–	–	–	–	–	✓
Configurable evaluation metrics	–	–	–	–	Partial	–	–	✓
Auto query + ground-truth generation	–	–	–	–	–	Partial	Partial	✓
Assertion-based fact checking	–	–	–	–	Partial	Partial	–	✓
User-level access control	–	–	–	Partial	Partial	Partial	✓	✓
Interactive debugging	–	–	–	–	–	–	–	✓
Pluggable LLM providers	–	Partial	–	–	✓	✓	Partial	✓
Sandboxed tool execution	–	Partial	–	–	–	–	–	✓

is scenario-specific: planning overhead pays off on the multi-week committee workload (Cambford), while a verbose-prompt small-model ReAct variant Pareto-dominates on cost across all tenants.

This design changes the role of a benchmark from a fixed leaderboard to an experimental sandbox. A researcher can generate a legal-services tenant and a software-incident tenant from the same pipeline, evaluate ReAct and plan-and-execute agents under identical access constraints, swap only the embedding model or only the citation judge, and then inspect the full tool trace when performance changes. Table 1 summarizes this shift relative to prior work.

The rest of the paper develops these ideas in order. Section 2 positions EABench relative to prior agent benchmarks and enterprise retrieval systems. Section 3 formalizes the end-to-end platform architecture, the configurable runtime, the data-generation pipeline, the evaluation-dataset generator, the multi-dimensional evaluation harness, and the interactive debugging interface. Section 4 describes the scenario families used to instantiate the benchmark. Section 5 specifies the experimental protocol supported by the framework, including cross-model evaluation, human validation, and ablation studies. Section 6 concludes with limitations and future directions.

2 Related Work

We review the literature across two primary axes: the evolution of agent benchmarking from general web environments to enterprise-specific tasks, and the enabling technologies behind EABench, including synthetic data generation, cognitive architectures, and automated evaluation metrics.

2.1 Agent Benchmarks

General-purpose benchmarks have been instrumental in driving rapid progress in LLM tool use. However, as discussed below, their reliance on static web environments and generic API puzzles limits their applicability to the messy, permission-gated reality of private organizational networks.

WebArena [25] provides a realistic web browsing environment with four web applications (shopping, reddit, GitLab, map) and 812 long-horizon tasks. While WebArena evaluates task completion in a realistic setting, its tasks are scoped to web interfaces and do not address enterprise knowledge retrieval. **WorkArena** [5] extends this approach to ServiceNow workflows, targeting enterprise service management tasks. However, its

data is limited to a single application and does not cover the richly inter-connected multi-source data of a real enterprise.

AgentBench [10] is a multi-dimensional benchmark spanning eight environments including OS, database, web browsing, and lateral reasoning tasks. It demonstrates that current LLMs struggle with multi-step tool-use tasks; however, its environments are largely synthetic or game-like rather than representative of knowledge-worker contexts.

ToolBench [15] focuses specifically on evaluating LLMs' ability to invoke real-world APIs from a large catalog of 16,000 APIs. While this directly tests tool-use, the tasks are API-centric rather than information-synthesis-centric, and there is no notion of user-level access control or data provenance.

τ-bench [21] introduces the concept of user simulation for interactive task evaluation in airline and retail domains. The approach of simulating realistic user interactions—rather than specifying fully deterministic tasks—is conceptually related to EABench's user-level evaluation mode, though τ-bench targets customer service scenarios rather than enterprise knowledge work.

Information Worker Benchmarks. While systems such as **OSWorld** [19] provide highly realistic tasks mapped to complex user interfaces, they remain fundamentally inflexible when modifications to the underlying environment or integrations of novel agent architectures are required. Conducting ablation studies within these robust ecosystems demands significant engineering overhead, restricting iterative cognitive architecture design.

Static vs. Dynamic Environments. GAIA-2 [4] and **SWE-Bench** [8] push the boundaries of general AI assistance by testing complex, real-world reasoning, multimodal tool use, and sophisticated software engineering tasks. However, these represent completely static and unalterable evaluation paradigms. Their rigid evaluation harnesses prevent isolating targeted variables—such as altering the underlying embedding model used in retrieval tools or limiting exposure to specific operational commands. Conversely, EABench explicitly addresses this via a deeply configurable design, granting researchers an open architectural playground to systematically investigate execution strategies across diverse scenario and runtime configurations.

2.2 Enterprise Agentic Benchmarks

Recognizing the limitations of general domains, recent works have begun simulating enterprise environments. While these benchmarks test multi-hop reasoning over corporate artifacts, they often rely on fixed, non-extensible datasets, leaving a critical gap for highly configurable, generative sandboxes.

DRBench [1] evaluates agents on open-ended, multi-hop deep research tasks that require synthesizing insight-driven reports from a mix of public web data and private company knowledge bases. Grounded in realistic user personas, DRBench spans a heterogeneous search space (including internal chats, cloud file systems, spreadsheets, and emails) and specifically scores an agent's ability to recall critical insights with proper source citations. Similarly, **HERB** [3] presents a benchmark containing 39,190 enterprise artifacts designed for evaluating deep search and retrieval-augmented generation over heterogeneous, interconnected data sources like documents, meeting transcripts, and Slack messages via synthetic workflows simulating product planning and support stages. While these benchmarks test multi-hop reasoning over enterprise artifacts, EABench distinguishes itself by generating a fully-coherent synthetic timeline, providing a fully configurable agent runtime (ReAct vs. Plan-and-Execute), and strictly verifying factuality via structural citation scoring combined with user-level access control.

Glean and similar enterprise search platforms combine vector search with access control lists (ACLs) to implement personalized search across connected SaaS applications. These systems illustrate the importance of user-context-aware retrieval—a feature EABench explicitly supports through its user-level indexing and filtering.

2.3 Synthetic Data Generation for Benchmarks

Real-world corporate data is scarce and constrained by extreme privacy bottlenecks. To overcome this, researchers have turned to LLM-driven synthetic generation. EABench extends these techniques beyond isolated QA pairs to construct fully interconnected, temporally coherent organizational ecosystems.

AgentInstruct [13] uses LLMs to generate instruction tuning data by transforming raw documents through a pipeline of multiple specialized agents. **FrontierMath** [6] demonstrates that LLM-generated problems can challenge state-of-the-art models. **WildChat** [23] collects naturally occurring LLM conversations in the wild.

EABench's approach differs from prior work in that it generates *structured organizational corpora* rather than isolated question-answer pairs. The generator produces inter-connected narrative artifacts (emails, meetings, files, chats) grounded in a shared storyline, enabling queries that require multi-hop reasoning across document types and temporal reasoning over event sequences.

2.4 LLM-as-a-Judge Evaluation

Evaluating open-ended, multi-step agent trajectories requires scalable metrics that transcend exact-string matching. LLM-as-a-Judge paradigms have emerged as a robust solution, which we adapt to perform granular citation verification and assertion checking while mitigating inherent evaluator biases.

Automated evaluation using LLMs as judges has gained significant traction following MT-Bench [24] and Chatbot Arena [2]. LLM judges show high correlation with human preferences at lower cost and enable evaluation of open-ended responses that resist automatic metrics.

EABench extends the LLM-as-a-Judge paradigm to *agentic evaluation*, where the judge must assess not only the final response but also the *reasoning process*—including whether cited sources are accurate, whether tool calls were appropriate, and whether the overall research plan was followed.

2.5 Cognitive Architectures and Agentic Workflows

The cognitive architecture of an agent fundamentally dictates its capability and operational ceiling. Recent advancements have synthesized rigid chain-of-thought prompting into broader structured agentic workflows, which emphasize iterative refinement, dynamic tool use, and reflection over single-shot generation. Within this landscape, we contrast single-pass reasoning loops with deliberative planning and broader workflow frameworks, all of which EABench treats as first-class, easily configurable strategies for applied scientists.

The ReAct paradigm introduced the concept of interleaving reasoning traces with tool invocations [22]. This approach demonstrated that explicitly reasoning about the next action before taking it substantially improves task performance on knowledge-intensive tasks compared to standard zero-shot execution. Plan-and-Solve agents build upon and extend this fundamental idea by explicitly separating the planning phase, which handles the generation of a high-level task decomposition, from the execution phase [18]. This deliberate

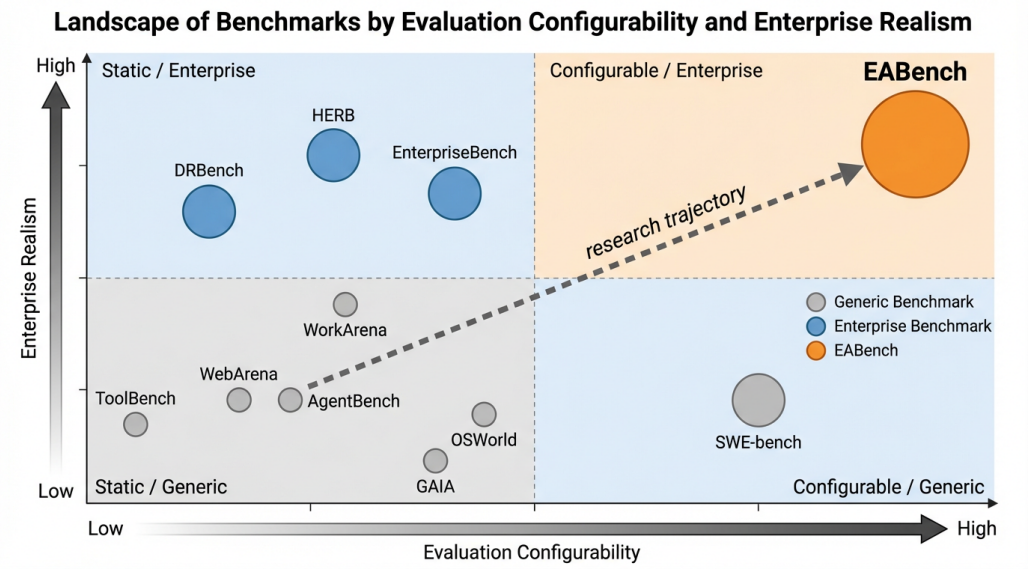

Fig. 2. Positioning of representative agent benchmarks along two axes: enterprise realism (y-axis) and evaluation configurability (x-axis). Bubble size reflects the number of first-class benchmark features supported (cf. Table 1). EABench uniquely occupies the top-right quadrant, combining scenario-aware organizational simulation with end-to-end configurability across all platform dimensions.

separation can significantly improve reliability on complex multi-step tasks by preventing the underlying agent from losing the contextual thread during long and intricate tool-use sequences.

Furthermore, modern agentic workflows have evolved these frameworks by incorporating reflection, multi-turn tool calling, and structured orchestration. These workflows shift the paradigm from monolithic text generation to a composite system where an agent iteratively critiques its intrinsic execution trajectory, adapts to newly retrieved information, and dynamically alters its multi-step plan. EABench natively implements these advanced paradigms as first-class, configurable execution strategies. This deep integration systematically enables the direct comparison of performance characteristics across distinct cognitive architectures and iterative workflows directly within evaluation tasks.

3 Methodology

We now describe the six components of the EABench platform in turn, covering the end-to-end architecture, the configurable runtime, the data-generation pipeline, the evaluation-dataset generator, the multi-dimensional evaluation harness, and the interactive debugging interface.

3.1 End-to-End Platform Architecture

EABench is organized as a single experimental pipeline rather than a loose collection of scripts. A scenario specification begins with a `StoryConfig`, which defines the company profile, timeline, employee count, and key events that drive narrative evolution. The tenant generator expands this specification into a multi-modal enterprise corpus containing users, emails, chats, meetings, and files. The same tenant is then consumed by

the agent runtime, which executes under a concrete user identity and records its full tool trajectory. Finally, the evaluation harness scores the resulting answer with both deterministic and judge-based metrics. Because each stage writes explicit artifacts to disk, the entire process is reproducible and inspectable.

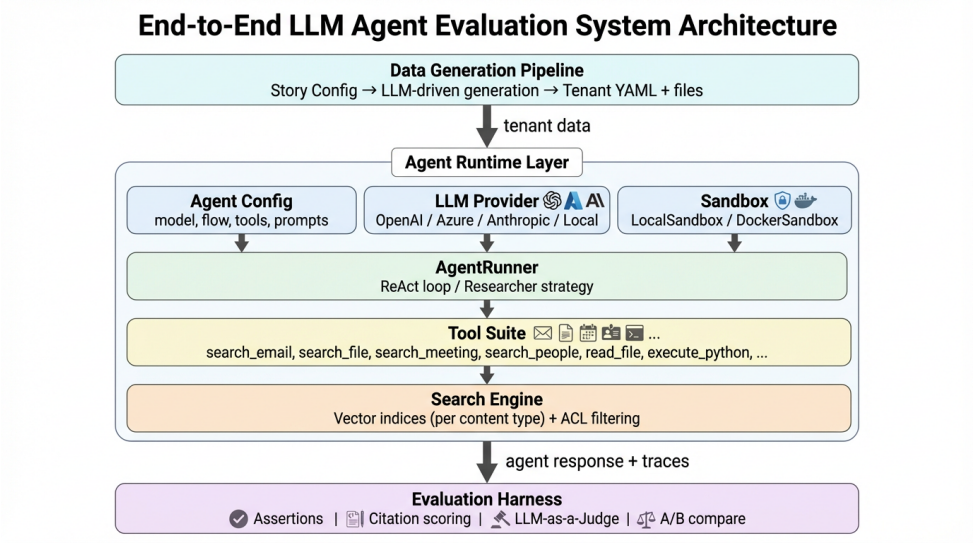

Fig. 3. End-to-end EABench architecture. The platform connects scenario specification, tenant generation, configurable runtime execution, evaluation-case generation, and interactive debugging in one reproducible workflow.

This architecture supports two complementary use modes. In batch mode, researchers generate a tenant, evaluate one or more agent configurations, and compare the resulting metric distributions. In interactive mode, they run the same runtime against the same tenant but inspect intermediate search results, tool calls, citations, and user context while debugging failure cases. The shared backend is important: debugging traces come from the exact runtime that produces benchmark results, so qualitative inspection and quantitative evaluation remain aligned.

The key systems claim of EABench is therefore not only that each subsystem is configurable, but that the interfaces between subsystems are stable and explicit. A change in the generator alters the tenant and the resulting query set; a change in the runtime alters the trajectory and answer; a change in the judge alters the scoring layer. By exposing these seams directly, EABench makes causal experimentation possible.

3.2 Configurable Agentic Runtime

3.2.1 YAML Configuration Schema. The runtime follows a configuration-as-code model in which each agent variant is represented by a single YAML file. This file specifies the reasoning model, embedding model, system prompt, optional planning prompt, per-tool query-analyzer prompts, tool exposure, and flow strategy. The design choice is deliberate: researchers should be able to version-control an agent definition, rerun it on a new tenant, and attribute behavioral changes to the configuration diff rather than to hidden code edits.

3.2.2 Cognitive Architectures. EABench currently exposes two cognitive architectures. Under the `react` strategy, the runtime loops through thought, tool call, observation, and response generation until the model

Table 2. Primary artifacts exchanged across the EABench pipeline.

Stage	Primary artifact	Role in the pipeline
Scenario specification	`StoryConfig` and prompt templates	Defines the enterprise domain, timeline, narrative drivers, and generator behavior
Tenant generation	`tenant.yaml`, `generation_log.json`, modality-specific YAML files, sandbox files	Materializes a coherent enterprise workspace with explicit provenance
Evaluation dataset generation	`eval_dataset_*.yaml`	Converts raw tenant history into query cases with `user_id`, `entity_list`, and natural-language assertions
Agent runtime	Response text, tool traces, and token-usage metrics	Executes a concrete agent configuration under a specific access context
Evaluation harness	Per-case JSON results and aggregate summaries	Produces assertion, citation, and operational diagnostics for comparison and analysis
Interactive debugging	Search evidence, tool-call logs, citation traces	Explains success and failure modes on individual cases without changing the runtime

```yaml
id: react-agent-v1
version: "1.0.0"
model:
  provider: azure
  name: gpt-4o
  parameters: {temperature: 0.7}
embedding:
  provider: azure
  model: text-embedding-ada-002
system_prompt: |
  You are an intelligent enterprise assistant.
  {user_profile}
query_analyzer_prompt:
  search_email: |
    Analyze the user's email query and return
    strategy, refined_query, and sender_name.
tools:
  definitions:
    - search_email
    - search_file
    - search_meeting
    - search_people
    - read_file
    - execute_python
flow:
  strategy: react
  max_turns: 5
```

Fig. 4. Condensed runtime configuration illustrating how model choice, prompt behavior, tool exposure, and flow strategy are all declared in YAML.

emits a final answer or reaches a hard turn limit. Under the `researcher` strategy, the runtime first invokes a planner to produce a structured plan and then executes that plan with the normal tool-enabled loop. The

Table 3. Six runtime dimensions that can be changed without source-code edits.

Dimension	Configuration field	Research question enabled
Reasoning model	`model.*`	How sensitive is performance to the underlying LLM family and decoding parameters?
Embedding model	`embedding.*`	How much of end-to-end performance is driven by retrieval quality rather than reasoning quality?
Cognitive architecture	`flow.strategy`	Does ReAct or plan-and-execute better handle multi-step enterprise tasks?
Global prompting	`system_prompt`, `planning_prompt`	How do role instructions and planning structure affect behavior and citation quality?
Tool exposure	`tools.definitions`	Which tools are necessary for each query type, and what are the failure modes when they are removed?
Sandbox implementation	tool dependencies and runtime backend	How do local, remote, or more restrictive execution environments change tool use?

Table 4. Comparison of the two built-in runtime strategies.

Strategy	First action	Typical strength	Typical failure mode
ReAct	Immediate tool-enabled reasoning	Fast response on local retrieval tasks	Can drift on long dependency chains or overlook needed decomposition
Researcher	Tool-free planning pass	Stronger structure on multi-step tasks	Higher token cost and possible over-planning on simple cases

second strategy adds overhead, but it is often better suited to multi-hop cases in which the agent must identify a dependency chain before any single tool call is obviously useful.

3.2.3 Multi-Provider LLM Support. The runtime separates the configuration schema from the provider implementation through a common LLM interface. Provider choice is driven by the YAML field `model.provider`, while credentials and deployment-specific settings are resolved from environment variables. This separation makes provider swapping operationally cheap: researchers can move between OpenAI, Azure-hosted deployments, or compatible local endpoints while preserving the rest of the runtime definition. That property is especially important for cross-model evaluations, because it prevents provider migration from becoming a hidden code change.

3.2.4 Enterprise Tool Suite and Search Infrastructure. The built-in tool suite combines semantic retrieval tools with sandboxed execution tools. Search tools cover emails, files, meetings, people, one-to-one chats, group chats, and channel messages. Execution tools cover file inspection, directory listing, shell execution, and Python execution. Tools are registered through a decorator-based registry that automatically exports JSON schemas for model-facing tool calling, while dependencies such as the sandbox, search engine, and query analyzer are injected by the runner. This keeps tool implementations modular and makes tool ablations straightforward: removing a tool from the YAML file is sufficient to change the agent's action space.

Retrieval itself is implemented as a set of separate vector indices rather than as one monolithic store. EABench indexes at least nine content collections, including file snippets, full file contents, emails, chats, group chats, channels, meeting metadata, meeting transcripts, and user profiles. Similarity is computed with

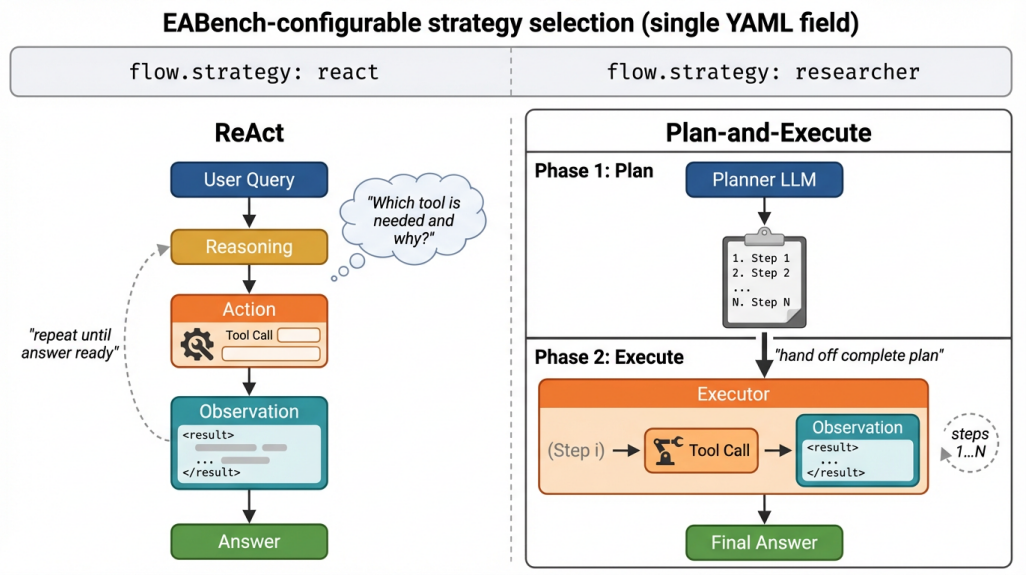

Fig. 5. The two cognitive architectures natively supported by EABench as hot-swappable `flow.strategy` values. ReAct (left) interleaves reasoning and acting in a tight feedback loop; Plan-and-Execute (right) separates a global planning phase from sequential step-wise execution, reducing context drift on long multi-hop tasks.

cosine similarity, and the resulting embeddings are cached on disk to avoid recomputation across repeated evaluations. The indices are filtered by user context before results are returned, which means the runtime and the evaluator both observe the same access-controlled search space.

3.2.5 Query Analyzer. An optional LLM-driven query analyzer sits between the user request and the retrieval tools. Instead of forwarding raw language directly to the vector index, the analyzer can refine a query, select a search strategy, or resolve a sender filter before retrieval occurs. Because its prompts are themselves configurable in YAML, EABench treats search refinement as a first-class experimental variable. Researchers can therefore study not only whether a search tool is useful, but whether specific decomposition heuristics or query rewrites materially improve retrieval fidelity.

3.3 Scenario-Aware Synthetic Data Generation

3.3.1 StoryConfig and Scenario Specification. Synthetic tenant generation begins with a `StoryConfig` that defines the company profile and the narrative arc the generator must realize. The configuration includes the company name, industry, company size, number of users, duration of the simulation, and a list of key events that anchor the narrative. This representation is compact enough for rapid scenario design yet expressive enough to vary organizational complexity and event density. The same generator can therefore instantiate a university administration scenario, a legal-services scenario, or a software-incident scenario without changing the code path.

3.3.2 Five-Stage Generation Pipeline. The generator executes in five stages. It first creates the directory scaffold and initializes modality-specific YAML files. It then generates the user roster in batches, using

Table 5. Core fields in the `StoryConfig` specification.

Field	Type	Purpose
`company_name`	string	Names the tenant and appears in prompts and output paths
`industry`	string	Conditions vocabulary, artifacts, and domain-specific events
`company_size`	string	Influences organizational breadth and communication patterns
`num_users`	int	Sets the size of the user roster and social graph
`duration_days`	int	Controls the temporal window over which history accumulates
`key_events`	list[string]	Defines narrative milestones that should recur across modalities
`description`	string	Supplies free-form scenario context to the generator prompts

diversity prompts to avoid name collisions and departmental imbalance. Next, it iterates day by day through the simulation, producing 3–5 narrative events per day while maintaining rolling history buffers. It materializes those events as emails, chats, meetings, and files, and it finally writes a complete `generation_log.json` that serves as the source material for evaluation-case generation. The pipeline is incremental by design: artifacts are appended to disk as they are generated, which supports large tenants without requiring all content to remain in memory.

Table 6. Five-stage tenant-generation process.

Stage	Name	Output and purpose
1	Directory scaffold and initialization	Creates tenant directories, empty modality YAML files, and a persistent workspace for generated documents
2	User generation with diversity control	Produces the employee roster, titles, departments, managers, skills, and group memberships while avoiding duplicates
3	Daily narrative generation	Expands key events into day-level story beats using recent-history and long-history buffers to preserve temporal coherence
4	Artifact materialization and cross-referencing	Generates emails, chats, meetings, and files whose content explicitly references shared events and decisions
5	Evaluation-log export	Serializes the full event history into `generation_log.json` for downstream query generation and auditability

3.3.3 Content Type Generation. Each artifact type is generated with a schema tailored to its role in enterprise communication. Emails are produced in two steps: a summary pass proposes sender, recipients, subject, and context, and a second pass expands that summary into realistic prose. Chats are likewise generated from conversation summaries into timestamped message sequences. Meetings produce three linked artifacts: metadata, transcript, and sidebar chat. Files are created as both metadata entries and concrete documents inside the sandboxed file tree. This staged process gives the generator enough structure to maintain consistency while still allowing varied surface realizations.

3.3.4 Narrative Coherence Mechanisms. Long-range coherence is preserved through explicit memory structures rather than through prompt length alone. A recent-history buffer stores the last several days of events verbatim, while a long-history buffer stores compressed summaries of earlier periods. Every seven simulated days, the recent buffer is summarized and appended to the long-term history. This mechanism allows later

generations to mention prior incidents, decisions, and unresolved tensions without forcing the model to ingest the entire raw history each day. Cross-document references are then encouraged directly in the prompts, so that an email can cite a meeting outcome, a file can summarize a discussion from chat, and a report can reference both.

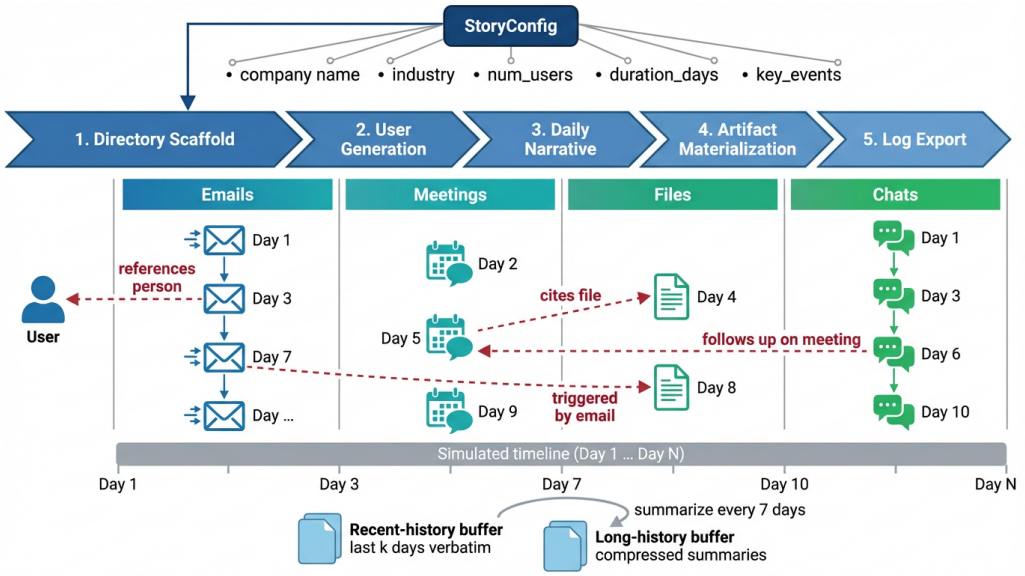

Fig. 6. EABench five-stage tenant-generation pipeline and inter-artifact linkage graph. A single `StoryConfig` drives the parallel creation of emails, meetings, files, and chats, which are connected by explicit cross-document references along a shared temporal timeline. These cross-references are what enable multi-hop and temporal reasoning evaluation cases.

3.4 Configurable Evaluation Dataset Generation

3.4.1 Three Query Types. EABench does not treat evaluation cases as fixed annotations authored by hand. Instead, it derives them automatically from the generated tenant history, producing three query types that cover retrieval, reasoning, and synthesis. Targeted artifact search cases ask the agent to locate a specific item or a small set of items using natural-language constraints on time, author, artifact type, or topic, without ever naming internal identifiers explicitly. Multi-hop reasoning cases require the agent to find information in one artifact in order to answer a question about another, or to trace a cross-artifact event chain across multiple modalities. Comprehensive report cases ask for a summary, timeline, or analysis of a specific topic grounded in a contiguous window of tenant activity. This tripartite design ensures that evaluation does not collapse to a single notion of success.

3.4.2 Ground Truth Association. Every generated case carries two forms of ground truth. The first is a structured `entity_list`, which records the concrete artifacts that a correct solution should retrieve. The second is a set of natural-language assertions that describe what the answer must contain. Each case is also bound to a specific `user_id`, which means the case is evaluated under an explicit access-control context

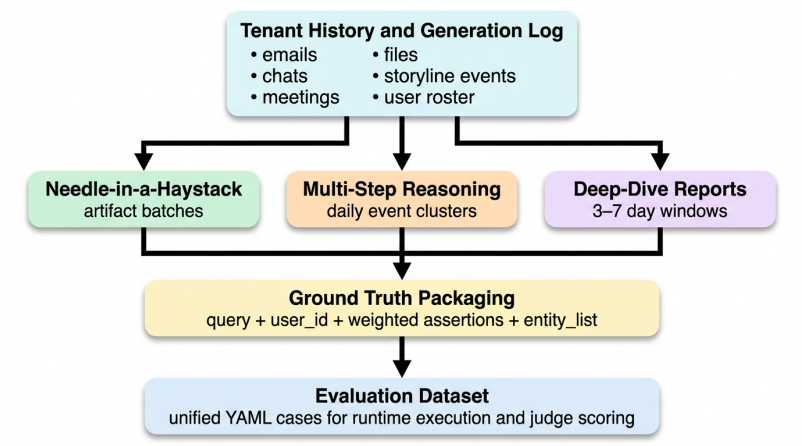

Fig. 7. Evaluation-case generation pipeline. The same tenant history is re-sliced into targeted artifact search, multi-hop reasoning, and comprehensive report cases, each with explicit user context and ground truth.

Table 7. Automatically generated query types in EABench.

Query type	Typical form	Primary benchmarked capability
Targeted artifact search	Natural-language query implying constraints on time, author, artifact type, or topic; targets a specific item or small result set without naming internal IDs (e.g., "Find emails sent by Alex about the Q3 budget in the last two weeks")	Targeted retrieval under natural phrasing and user-level access control
Multi-hop reasoning	Query that requires finding information in one artifact to answer a question about another, or tracing a cross-artifact event chain (e.g., "Who attended the meeting discussed in the budget email from March, and what did they decide?")	Cross-modal reference resolution and dependency-chain reasoning
Comprehensive report	Request for a summary, timeline, or analysis of a specific topic grounded in a contiguous window of tenant activity (e.g., "Summarize all decisions related to the server migration from March 1 to March 15")	Long-form synthesis, temporal organization, and citation-grounded summarization

rather than against a globally visible corpus. This design is important because it distinguishes "the answer is wrong" from "the answer is unavailable to this user."

3.4.3 Assertion Generation for Fact Checking. Assertions are generated automatically from the same context used to formulate the query. They are written in natural language and focus on factual obligations rather than stylistic preferences. A targeted artifact search case might require the answer to name the correct sender and subject line, while a comprehensive report case might require the answer to mention the triggering event,

the main decision, and the relevant date range. This representation makes evaluation robust to paraphrase while still preserving strong factual expectations.

3.4.4 Realism Constraints. The prompts that generate cases enforce several realism constraints. Queries never mention internal artifact identifiers directly. User identities are assigned plausibly, such as recipients for emails or attendees for meetings. Multi-hop reasoning cases are only produced when the source context contains a realizable cross-artifact dependency chain, and comprehensive report cases are grounded in contiguous event windows so that the requested narrative exists in the tenant history. These constraints matter because they prevent the generator from creating benchmark cases that are formally valid but operationally unnatural.

3.4.5 Generation Pipeline. Case generation proceeds in three passes over the tenant history. The first pass samples individual artifact summaries and generates targeted artifact search queries with implicit attribute constraints. The second pass groups events by day and produces multi-hop reasoning cases only when the context contains a realizable cross-artifact dependency chain. The third pass samples contiguous multi-day event windows and generates comprehensive report requests over the resulting clusters. The target split is approximately 40% targeted artifact search, 40% multi-hop reasoning, and 20% comprehensive report; search and multi-hop cases are more numerous because they offer finer-grained diagnostic signal per case, while report cases require a larger context window and are generated at lower frequency per history segment. Observed splits deviate slightly from this target because each generator produces variable-sized batches. By writing all cases to a single evaluation dataset with shared structure, EABench ensures that retrieval, reasoning, and report synthesis can be evaluated side by side under the same runtime and judge configuration.

3.5 Multi-Dimensional Evaluation Harness

3.5.1 Judge Configuration via YAML. The evaluation harness is configured independently from the runtime under test. A separate YAML file specifies the judge model, decoding parameters, and prompt templates for each scoring function. This separation is intentional. It allows a stronger or more stable model to serve as the judge, and it lets researchers redefine the evaluation criteria without editing the evaluation code. In practice, we recommend using a deterministic judge configuration with temperature fixed at zero so that repeated runs remain comparable.

3.5.2 Configurable Evaluation Metrics. EABench separates the *harness logic* from the *evaluation criteria*. Researchers specify what to measure entirely through configuration; no code changes are required to add, replace, or retune a metric.

Judge-based metrics are defined by prompt templates in the judge YAML file (Figure 8). Each template receives the agent's query, response, and tool trace, and returns a score in $[0, 1]$ with an explanation. Any number of judge prompts can be registered; common patterns include:

- **Factual assertion checking.** A prompt that verifies whether each gold-standard assertion in the evaluation case is satisfied by the agent's response. The per-case score is a pass fraction over all assertions.

```yaml
name: "Default Judge Prompts"
model:
  provider: azure
  name: gpt-4o
  parameters: {temperature: 0.0}
prompts:
  assertion_check: |
    Verify whether the response satisfies each assertion.
  citation_relevance: |
    Judge the relevance of tool results to the query.
  response_citation: |
    Judge whether cited IDs are real, relevant, and
    grounded in the tool trace.
```

Fig. 8. Condensed judge configuration illustrating independent control of the evaluation model and prompt templates.

- **Retrieval quality.** A prompt that jointly evaluates tool selection, evidence relevance, and response grounding—checking whether the agent's answer faithfully reflects what was retrieved rather than hallucinating.
- **Citation grounding.** A prompt that inspects the response's references section: whether it exists, whether cited artifact IDs appear in the tool trace, and whether inline citation markers are present.
- **Derived aggregates.** Scores derived from two or more judge outputs—e.g., the mean of retrieval quality and citation grounding—to summarize end-to-end evidence handling.

Deterministic diagnostics are extracted from the tool trace and response text without an LLM call. They are always computed regardless of judge configuration and serve as interpretable proxies for agent behavior:

- **Retrieved-item count.** Total evidence items returned across all search tool calls.
- **Response citation count.** Number of structured reference entries emitted in the final answer.
- **Token usage, tool-call count.** Operational cost and trajectory efficiency.

Pass/fail threshold. Whether a case is marked as passed is controlled by threshold parameters applied to any subset of the judge scores. Setting tighter thresholds raises the bar for correctness and grounding; researchers can calibrate these to match the compliance requirements of their target deployment context.

3.5.3 Pass/Fail Criteria. A case is marked as passed when both of the following conditions hold simultaneously:

- **Assertion score** ≥ 0.75**.** At least 75% of the natural-language assertions are satisfied. This threshold reflects enterprise contexts in which a response missing more than one in four factual requirements is operationally unacceptable.
- **Response-citation quality** ≥ 0.7**.** The final answer must include a sufficiently grounded references section. This gate ensures that a plausible-sounding response is still marked as failed if it cites hallucinated artifacts.

Table 8. Metrics reported by the EABench evaluation harness.

Metric	Type	Interpretation
Assertion score	Judge-based	Fraction of natural-language assertions satisfied by the final answer
Tool search result relevance	Judge-based	Whether the selected tools and retrieved evidence were appropriate and faithfully reflected in the answer
Response citation score	Judge-based	Whether the final answer cites real, relevant artifacts and includes grounded references
Combined citation score	Derived	Mean of tool relevance and response citation scores
Tool search result number	Deterministic	Number of retrieved evidence items surfaced across search tool calls
Response citation number	Deterministic	Number of references emitted by the agent in the final answer
Token usage, tool-call count	Deterministic	Operational cost and efficiency of the agent trajectory

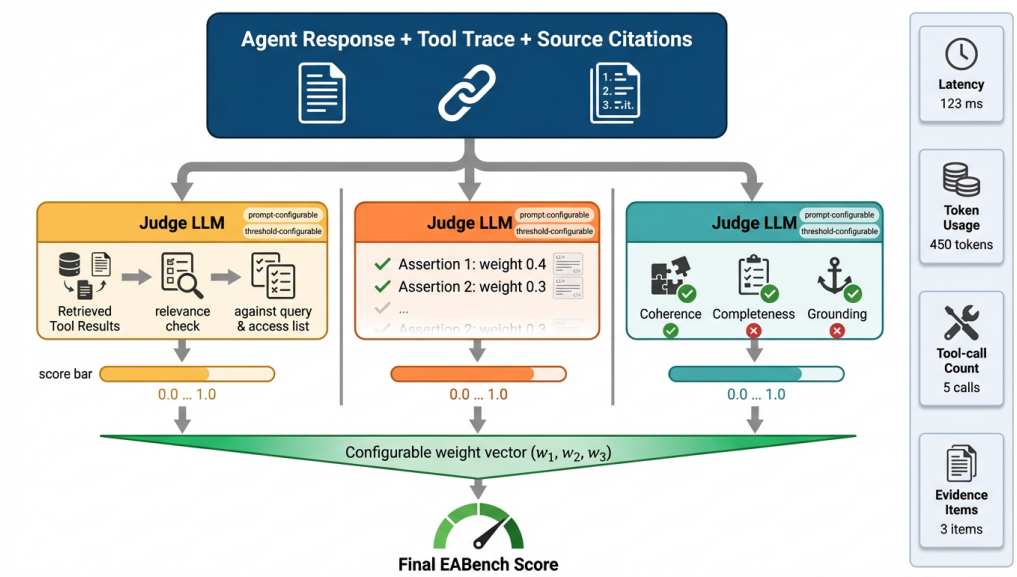

Fig. 9. EABench multi-dimensional evaluation harness. Two independently configurable LLM-judge tracks score tool-search relevance and response-citation quality; their equal-weight average forms the combined citation score. The assertion score is computed by a third judge call. All three judge scores are reported alongside deterministic diagnostics for operational profiling.

This dual criterion encodes an important design principle: correctness and grounding are jointly necessary for enterprise-agent success. An agent that satisfies all assertions but fabricates its citations fails the grounding gate; an agent with a well-formed references section that misses key facts fails the correctness gate.

We do not currently report a human-annotator validation study of assertion grounding, satisfiability, and inter-rater agreement; the assertion-generation pipeline is instead validated indirectly through judge–judge agreement checks and through a post-hoc keyword-based audit of query-type coverage (Section 5). We treat a full human-annotation study as an open validation task (Section 5.5).

3.5.4 Side-by-Side Comparison and Diagnostics. For comparative studies, EABench can evaluate two agent configurations on the same case and ask a judge model to declare a winner. These pairwise judgments support win-rate analyses and Bradley–Terry modeling, while the underlying per-case metric vectors support paired tests such as the Wilcoxon signed-rank test. Because the result structure stores the full response, tool trace, assertion outcomes, and citation logs, comparative findings are accompanied by diagnostics that explain why one agent won rather than only whether it won.

3.5.5 User-Level Experience Simulation. Evaluation is always grounded in a requesting user context. Before each case is run, the search engine is bound to `case.user_id`, and access-control rules are enforced consistently across search results, citation verification, and debugging views. This design allows the same tenant to support engineer, manager, and contractor personas without maintaining separate corpora, and it turns access control into a benchmarked capability rather than an external precondition.

3.6 Interactive Debugging and Chat Interface

3.6.1 Live Chat Interface. Batch evaluation is necessary for aggregate comparison, but it is insufficient for rapid iteration on agent design. EABench therefore ships an interactive Streamlit application (Figure 10) in which the same runtime used for evaluation can be exercised through a live chat interface. A researcher selects an agent configuration, a tenant dataset, and a user persona from the sidebar, then issues free-form queries to inspect the agent's behavior under that persona's access scope. The main panel displays the agent's structured response with inline citation markers and a references section listing source artifacts with metadata (type, author, date, entity ID). This mode is especially valuable during prompt engineering and tool-ablation studies, because it shortens the loop between a configuration change and a concrete behavioral observation.

3.6.2 Debugging Facilities. The interactive layer exposes the pieces of evidence that matter for debugging enterprise agents. The sidebar navigation provides four views: *Chat* for live query exploration, *Evaluation* for batch result inspection, *Side-by-Side Comparison* for contrasting agent configurations on the same query, and *Data Generator* for creating new tenant datasets. In the Chat view, the agent's response includes inline citation markers that link to a References section with full artifact metadata—type, author, date, and entity ID—enabling a researcher to verify each cited source against the tenant corpus. The Login panel displays the active user's role and ID, making access-scope constraints transparent. These views make it possible to distinguish several common failure modes: the agent may choose the wrong tool, retrieve weak evidence, lose track of a multi-hop chain, or cite the right artifact incorrectly. Because these signals are surfaced directly, EABench supports debugging by diagnosis rather than by prompt trial-and-error alone.

3.6.3 User Experience Testing. The interactive workflow is also useful for evaluating enterprise usability beyond aggregate benchmark scores. Researchers can replay the same case under multiple personas, inspect which artifacts become visible or hidden, and compare how different runtime configurations explain uncertainty to the user. In this sense, the interactive interface serves as a bridge between formal benchmarking and deployment-oriented validation: it preserves reproducibility while surfacing the qualitative details that determine whether an enterprise agent is trustworthy in practice.

Table 9. Debugging surfaces exposed by the interactive workflow.

Surface	What it reveals
Agent & tenant selectors	Switch agent configuration and corpus without restarting
User persona login	Role, user ID, and access scope active for the current session
Inline citations	Numbered markers in the response linking to specific source artifacts
References panel	Full metadata (type, author, date, entity ID) for each cited artifact
Side-by-side comparison	Contrast two agent configurations on the same query and persona
Batch evaluation view	Aggregate metrics and per-case drill-down for completed eval runs

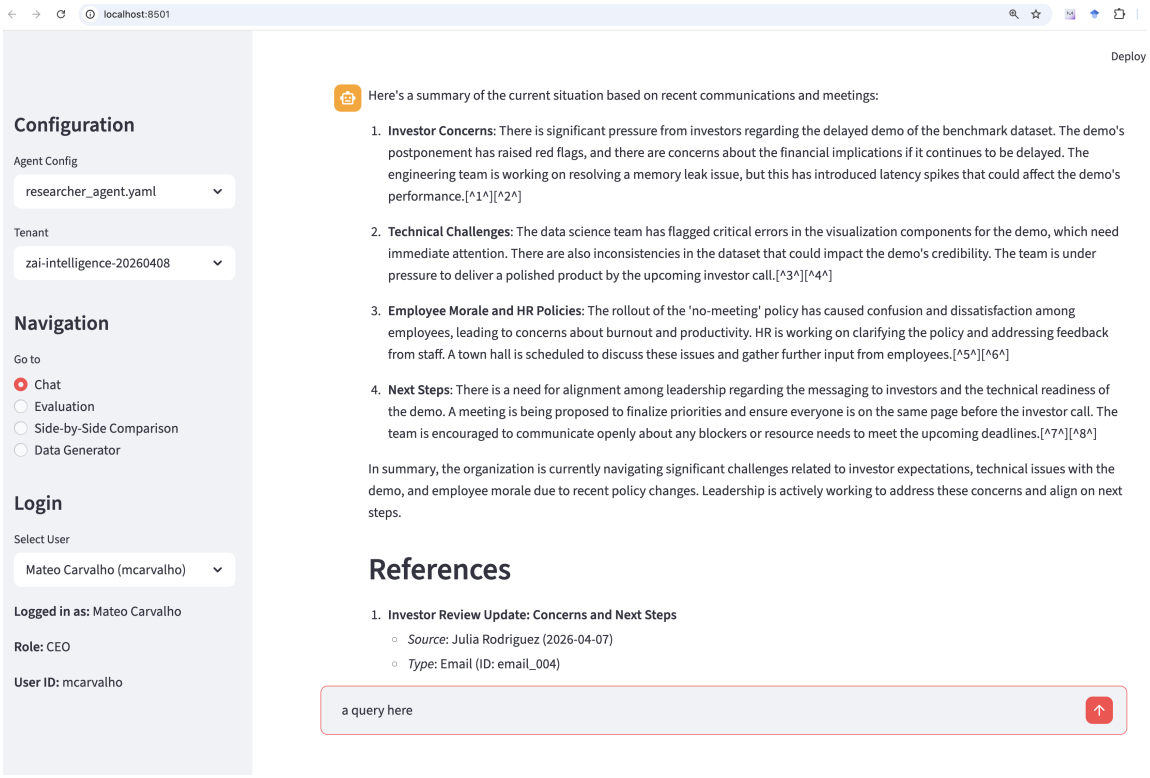

Fig. 10. EABench interactive chat interface. The left sidebar provides configuration controls (agent strategy, tenant, and user persona); the main panel displays the agent's structured response with inline citation markers and a references section listing source artifacts with metadata. A query input box at the bottom enables iterative exploration under the selected persona.

4 Benchmark Scenarios and Instantiations

To demonstrate how EABench instantiates distinct enterprise workloads without changing the underlying codebase, we define three scenario families that vary organizational structure, confidentiality boundaries, and temporal reasoning demands. The purpose of these scenarios is not merely to populate different datasets; each scenario is designed to stress a different slice of agent capability, including access-aware retrieval,

multi-hop reasoning across modalities, and long-form synthesis over multi-day windows. Table 10 summarizes the intended stressors.

Table 10. Scenario families used to instantiate EABench and the benchmark capabilities they emphasize.

Scenario	Primary stressor	Dominant artifacts	Representative evaluation focus
Education	Policy evolution across administrative silos	Committee emails, meeting transcripts, planning documents, service-status updates	Timeline reconstruction for curriculum changes, attribution of administrative decisions, and comprehensive reports spanning outages and policy revisions
Law firm	Strict confidentiality and role-dependent visibility	Client briefs, internal legal memoranda, partner chats, case meetings	Access-control compliance, privilege-sensitive retrieval, and citation-grounded answers under consultant and attorney personas
Software company	Cross-modal incident response and product execution	Incident reports, engineering chats, sprint meetings, launch documents, customer escalation emails	Multi-hop tracing from symptom to root cause, tool-heavy search trajectories, and report generation over short operational windows

4.1 Education: University Administrators and Faculty

The education scenario models a university-like environment in which decisions emerge through committee processes rather than single-owner workflows. This structure produces queries that require the agent to connect policy drafts, meeting discussions, and follow-up emails over multi-week windows. It is therefore particularly useful for evaluating comprehensive report generation and temporally grounded retrieval, because the correct answer often depends on reconstructing how a proposal evolved across multiple administrative groups rather than locating a single decisive artifact.

4.2 Law Firm: Consultants and Attorneys

The law-firm scenario emphasizes confidentiality and sharply differentiated access rights. Here the central question is not only whether the agent can retrieve relevant material, but whether it can do so while respecting the requesting user's role. This scenario is especially effective for evaluating user-level experience simulation: the same natural-language request may be answerable for a partner, partially answerable for a staff attorney, and intentionally blocked for a consultant. As a result, the scenario exercises both the retrieval stack and the policy layer that sits on top of it.

4.3 Software Company: SDEs and Managers

The software-company scenario concentrates dense cross-references into short operational bursts such as outages, launch reviews, and rollback decisions. It is the most tool-intensive scenario because answers frequently require the agent to combine people search, chat retrieval, meeting lookup, and file inspection inside the same trajectory. This makes it a strong stress test for the difference between ReAct and plan-and-execute agents: the former can react quickly to local evidence, while the latter benefits when a multi-step dependency chain must be planned before execution.

Table 11. Artifact composition and evaluation case counts for the three EABench tenants. For chats and group chats, the count reflects distinct conversation threads rather than individual messages. Evaluation cases span targeted artifact search, multi-hop reasoning, and comprehensive report query types (see Section 5.1.2).

Tenant	Domain	Users	Emails	DMs	Group Chats	Meetings	Files	Eval Cases		
								Tgt. Search	Multi-Hop	Compreh.
Bertrand & Co.	Legal	41	132	56	118	69	70	80	43	39
Univ. of Cambford	Education	62	120	41	118	70	70	80	44	38
ZAI Intelligence	AI / SW	165	298	140	301	164	164	79	82	40
Total		268	550	237	537	303	304	239	169	117

4.4 Dataset Statistics

We instantiated EABench across three concrete tenant organizations spanning distinct enterprise domains: a boutique law firm (*Bertrand & Co.*, 41 users), a mid-size research university (*University of Cambford*, 62 users), and a fast-growing AI startup (*ZAI Intelligence*, 165 users). Each tenant was generated independently by the same GPT-4o-based pipeline described in Section 3.3, parameterized with a domain-specific company description, a set of key narrative events, and an organizational size. The staged generator ensures that artifacts within a single simulated day are mutually coherent—an email thread's content is consistent with the meeting that prompted it—while still allowing realistic variation across days.

Table 11 summarizes the resulting artifact counts and evaluation-case distribution. For each tenant, three evaluation query generators—targeted artifact search, multi-hop reasoning, and comprehensive report—produce structured test cases in a separate pass that examines the completed corpus and constructs questions, gold assertions, and entity lists against it.

The three tenants vary substantially in scale: ZAI Intelligence is roughly four times larger than Bertrand & Co. across every artifact category, and its eval-case count (201) is proportionally expanded to maintain per-tenant statistical power. This heterogeneity is intentional—EABench is designed to stress-test agents under differing information densities rather than to standardize them away.

The query-type distribution follows the pipeline's target split of approximately 40/40/20 (targeted artifact search / multi-hop reasoning / comprehensive report). The actual ratios deviate slightly because each generator produces variable-sized batches. Targeted artifact search queries each reference exactly one gold entity, multi-hop reasoning queries reference 2–5 entities from different modalities, and comprehensive report queries reference 3–15 entities spanning multiple artifact types and dates.

Table 12 reveals a clear complexity gradient across query types. Targeted artifact search queries average 2.2–2.6 assertions grounded in a single entity, making them the simplest evaluation targets. Multi-hop reasoning queries maintain a similar assertion count but require cross-referencing 2–5 entities from different communication modalities. Comprehensive report queries demand the most complex synthesis: 3–4 assertions each backed by 3–15 source entities spanning emails, chats, meetings, and files. This structured complexity scaling ensures that the evaluation set probes agent capabilities at multiple difficulty levels.

4.5 Communication Network Properties

To characterize inter-personal communication structure, we extract a directed multigraph for each tenant in which nodes are users and edges represent recorded communicative acts across five channel types:

Table 12. Ground-truth evaluation complexity by query type and tenant. "Assertions" are gold-standard factual statements the agent's response must satisfy; "Entities" are the ground-truth artifacts (emails, chats, meetings, files) required to answer the query.

		Assertions per Case			Entities per Case		
Tenant	**Query Type**	**Mean**	**Med.**	**Range**	**Mean**	**Med.**	**Range**
Bertrand	Targeted Artifact Search	2.4	2	2–4	1.0	1	1–1
	Multi-Hop Reasoning	2.3	2	2–3	2.7	2	2–5
	Comprehensive Report	3.2	3	3–4	5.3	5	4–10
Cambford	Targeted Artifact Search	2.6	3	2–3	1.0	1	0–1
	Multi-Hop Reasoning	2.8	3	2–4	3.1	3	2–4
	Comprehensive Report	3.2	3	3–4	5.9	5	3–15
ZAI	Targeted Artifact Search	2.2	2	2–3	1.0	1	1–1
	Multi-Hop Reasoning	2.3	2	2–5	2.4	2	2–5
	Comprehensive Report	3.2	3	3–4	6.0	5	3–11

email (*from*→*to*), email CC (*from*→*cc*), direct-message chat (*sender*→*recipient*), group chat (*sender*→all participants), and meeting (pairwise edges among all attendees plus *organizer*→*attendee* edges). Table 13 summarizes the structural and temporal metrics we report; Table 14 then gives the per-tenant values.

Table 13. Definitions of the dataset characterization metrics used in Sections 4.5 and 4.6. $G = (V, E)$ denotes the directed communication multigraph for a tenant; $\tau_1, \tau_2, \ldots, \tau_n$ denote the inter-event time (IET) sequence on a given channel.

Metric	**Definition**	**Interpretation**
Structural (communication network)		
$\|V\|$, $\|E\|$	Counts of nodes (users) and directed edges (communicative acts).	Scale of the organizational graph.
Density	$\|E\|/(\|V\|(\|V\|-1))$.	Fraction of possible directed ties realized; higher in small, tight-knit organizations.
Diameter	Longest shortest path over all connected node pairs.	Worst-case hop count for information to traverse the organization.
Avg. Path	Mean shortest-path length over all connected pairs.	Typical separation between two employees.
Reciprocity	Fraction of directed edges (u, v) for which $(v, u) \in E$.	Tendency for communications to be returned; high for email, zero for chat by construction.
Transitivity	Global clustering coefficient: fraction of connected triples that form triangles.	Prevalence of "friend-of-friend" closure.
γ, R^2	Exponent and goodness-of-fit of the power-law $P(k) \sim k^{-\gamma}$ fit to the in-degree CCDF.	Low γ and high R^2 indicate a heavy-tailed, hub-and-spoke structure.
Temporal (per-channel inter-event times)		
Events	Number of timestamped events in the channel.	Sample size for IET statistics.
B	$(\sigma_\tau - \mu_\tau)/(\sigma_\tau + \mu_\tau)$ [7].	$B \approx 0$: Poisson/regular; $B > 0$: bursty clustering; $B < 0$: anti-bursty.
M	Lag-1 autocorrelation of the IET sequence [7].	$M > 0$: long gaps follow long gaps (multi-scale clustering); $M \approx 0$: memoryless.
α	Exponent of the power-law fit $P(\tau) \sim \tau^{-\alpha}$ to the IET distribution.	Lower α means heavier tail (more extreme silent gaps).
Med. IET	Median inter-event time (seconds).	Typical temporal spacing; small for chat, large for email.

Table 14. Communication network topology for each tenant. See Table 13 for metric definitions.

Tenant	$\|V\|$	$\|E\|$	**Density**	**Diam.**	**Avg. Path**	**Recip.**	**Trans.**	γ	R^2
Bertrand	41	322	0.196	3	1.71	0.286	0.486	0.36	0.29
Cambford	62	519	0.137	4	1.89	0.343	0.448	0.68	0.42
ZAI	165	1,956	0.072	4	2.10	0.314	0.367	0.66	0.59

These metrics reflect realistic organizational archetypes. Bertrand & Co., the smallest organization, is the densest (0.196) and has the smallest diameter (3), consistent with a tight-knit professional-services firm where most colleagues interact directly. ZAI Intelligence, the largest, exhibits a sparse, long-tail structure (density 0.072, 1,956 edges among 165 nodes) typical of rapidly growing technology companies with departmental silos. The University of Cambford occupies an intermediate position with the highest reciprocity (0.343), reflecting the collaborative back-and-forth characteristic of academic committee work. Transitivity is highest for Bertrand (0.486), where shared client matters create strong triadic closure, and lowest for ZAI (0.367), where cross-functional interaction is sparser.

The in-degree distributions follow a heavy-tailed pattern. We fit a power law $P(k) \sim k^{-\gamma}$ to the complementary cumulative distribution of in-degrees and compare the empirical graph against a synthetic Barabási–Albert (BA) preferential-attachment graph of the same size. The low γ values (0.36–0.68) confirm that all three networks exhibit right-skewed degree distributions rather than Poisson-like regularity. The R^2 values increase with tenant size (0.29 for Bertrand, 0.59 for ZAI), indicating that the power-law fit improves as the number of nodes permits a wider degree range. In each case, the maximum in-degree substantially exceeds the BA model's prediction (e.g., 99 vs. 72 for ZAI, 43 vs. 34 for Cambford), indicating that the generated organizations produce realistic "hub" individuals—such as executives, team leads, or central administrative staff—whose communication connectivity is disproportionately high.

Per-channel decomposition reveals further structural realism. Email communication has reciprocity between 0.48 and 0.73 across tenants, reflecting the natural tendency for emails to receive replies. Chat messages are entirely non-reciprocal at the edge level (0.0), since each message is a directed act from sender to recipient with no guaranteed return in the same thread. Meeting edges, constructed from attendee co-presence, are fully reciprocal by construction (1.0). This channel-level heterogeneity is important for evaluation: an agent that reasons about "who communicated with whom" must integrate evidence from channels with qualitatively different relational semantics.

4.6 Temporal Communication Patterns

Realistic enterprise datasets exhibit bursty temporal structure: communication tends to cluster around deadlines, incidents, or scheduled events rather than arriving as a uniform Poisson process. We quantify this using the burstiness parameter $B = (\sigma_\tau - \mu_\tau)/(\sigma_\tau + \mu_\tau)$, where μ_τ and σ_τ are the mean and standard deviation of the inter-event time (IET) sequence for a given communication channel [7]. Values of B near zero indicate exponential (Poisson) inter-arrival times, while $B > 0$ indicates bursty clustering. We additionally report the memory coefficient M [7], which measures the correlation between consecutive IETs: $M > 0$ indicates that long gaps tend to follow long gaps (temporal clustering at multiple scales), while $M \approx 0$ indicates uncorrelated consecutive intervals.

Table 15. Inter-event time (IET) statistics per communication channel and tenant. See Table 13 for metric definitions. Meeting channels are omitted for brevity ($|B| < 0.01$ in all tenants except ZAI, where $B = 0.23$ with $M = 0.19$).

Tenant	Channel	Events	B	M	α	Med. IET (s)
Bertrand	Email	132	−0.051	−0.054	2.00	81,450
	Chat (1:1)	1,022	+0.617	−0.055	0.33	72
	Group Chat	2,291	+0.560	−0.078	0.36	120
Cambford	Email	120	+0.065	−0.045	1.49	85,500
	Chat (1:1)	732	+0.640	−0.048	0.35	90
	Group Chat	2,075	+0.575	−0.072	0.34	110
ZAI	Email	298	+0.168	+0.395	1.28	82,800
	Chat (1:1)	2,581	+0.658	−0.042	0.33	77
	Group Chat	6,033	+0.614	−0.055	0.31	150

The results validate that EABench's generation pipeline reproduces two hallmarks of real organizational communication. First, email arrivals are near-Poisson ($B \approx 0$) across all three tenants, with high α values (1.28–2.00) indicating that inter-arrival times decay rapidly—consistent with the scheduled nature of formal correspondence. Second, chat and group-chat streams are highly bursty ($B \in [0.56, 0.66]$) with low power-law exponents ($\alpha \in [0.31, 0.36]$), confirming heavy-tailed inter-event time distributions where short bursts of rapid-fire messages are separated by long silences. This pattern matches empirical measurements of instant-messaging platforms in organizational settings [7].

The memory coefficients are near zero for most channel–tenant pairs ($M \in [-0.08, 0.0]$), indicating that the IET process is approximately memoryless at the single-channel level—as expected when daily narrative generation does not explicitly model autoregressive temporal dependence within a channel. The notable exception is ZAI's email channel ($M = +0.40$), where the organizational scale produces enough correlated event clusters (e.g., multi-day incident threads, product-launch email bursts) to induce measurable serial dependence.

This temporal structure directly impacts retrieval difficulty: an agent must reason about *when* information was generated, not only *what* it contains, because a correct multi-hop answer often depends on reconstructing an event sequence from artifacts whose timestamps are concentrated in bursty clusters rather than distributed uniformly.

5 Experiments

EABench is designed to support controlled experimentation rather than a single fixed leaderboard. We organize this section around the experimental protocol: a description of the setup and metric definitions, followed by empirical results across four agent configurations and three tenant scenarios, ablation studies isolating the contribution of architectural and prompt-engineering choices, and an analysis of the cost-effectiveness frontier. Table 16 summarizes the core experimental design.

5.1 Experimental Setup

We instantiate the three scenario families from Section 4 as concrete tenants (Bertrand & Co., University of Cambford, and ZAI Intelligence) and evaluate four agent configurations that span a spectrum from

Table 16. Core experimental protocol supported by EABench.

Axis	Controlled variables	Primary outputs	Purpose
Cross-model evaluation	Tenant generator, runtime model, judge model, flow strategy	Pass rate, assertion score, citation score, win rate	Detect self-preference bias; verify ranking stability across generator–judge combinations
Ablation studies	Architecture (ReAct vs. plan-and-execute), prompt verbosity, tool configuration	Metric deltas per agent–tenant cell	Attribute performance to specific platform components
Scenario analysis	Industry template, company size, event density, persona mix	Per-scenario score distribution and failure taxonomy	Determine which scenario characteristics most affect retrieval, reasoning, and citation

minimal reactive reasoning to deliberative planning; the precise model and prompt assignments are listed in Section 5.1.1 and Table 17. The automated judge is GPT-4o, issuing one call per gold assertion to check satisfaction plus two calls per case for tool-search relevance and response-citation quality. Each agent is evaluated on all 525 cases (162 per small tenant, 201 for ZAI) covering the three query types defined in Section 3.4. We record both judge-based quality metrics and deterministic operational diagnostics—token usage, tool-call count, and retrieval volume—to distinguish capability failures from operational inefficiency.

5.1.1 Agent Configurations. We evaluate four agent configurations whose designs span a spectrum from minimal reactive reasoning to deliberative multi-step planning. All four share the same embedding model (`text-embedding-ada-002`) and use GPT-4o-family models via Azure OpenAI: ReAct-v1 and ReAct-v2 use GPT-4o, while ReAct-v3 and Researcher use GPT-4o-mini. This design decouples three axes—architecture (ReAct vs. plan-and-execute), prompt engineering (verbose vs. concise), and model size (GPT-4o vs. GPT-4o-mini)—so that pairwise comparisons can attribute gains to individual factors rather than to their confound.

Table 17. Configuration summary for the four evaluated agents. "Planning model" refers to the LLM used for the optional pre-execution planning step; for Researcher, the same model is used for both planning and execution. Prompt size is the approximate system-prompt token count. Avg. tool calls is the empirical mean across all evaluation cases.

Agent	Architecture	Model	Planning Model	Prompt Size	Avg. Tools
ReAct-v1	ReAct	GPT-4o	—	∼3k tok	1.70
ReAct-v2	ReAct	GPT-4o	—	∼1.5k tok	1.70
ReAct-v3	ReAct	GPT-4o-mini	—	∼3k tok	1.23
Researcher	Plan + Execute	GPT-4o-mini	GPT-4o-mini	∼3k tok	4.85

ReAct-v1 implements a standard ReAct loop [22] with a verbose system prompt (∼3,000 tokens) containing eight fully worked Thought→Tool examples covering people resolution, pronoun-to-ID disambiguation, multi-query decomposition, and citation formatting. Section 5.4.1 analyses which of these worked demonstrations drive the observed performance gap against ReAct-v2.

ReAct-v2 shares the same single-pass ReAct loop but condenses the system prompt to ∼1,500 tokens across five terse bullet-style examples. The instructions cover the same topics, but replace the extended worked demonstrations with one-line directives.

ReAct-v3 is a controlled ablation that shares ReAct-v1's architecture and verbose prompt verbatim but swaps the backbone model from GPT-4o to GPT-4o-mini. This isolates the effect of model size at fixed architecture and prompt, which in turn lets the ReAct-v3 ↔ Researcher comparison isolate the planning-stage effect at fixed model.

Researcher instantiates the plan-and-execute strategy introduced in Section 3.2: GPT-4o-mini first produces a structured research plan, then the same model executes that plan step by step using the full tool set. The planning prompt explicitly requires four capabilities—(i) multi-hop decomposition into dependency chains, (ii) correlated search across emails, chats, meetings, and files, (iii) mandatory name-to-ID resolution via `search_people` before any content search, and (iv) step-by-step ordering so that the output of each step feeds the next—and instructs the planner to output only the plan, not to execute it. As a result, Researcher retrieves substantially more evidence per case and consumes roughly an order of magnitude more prompt tokens than the reactive agents (quantified in Section 5.3.1). Section 5.3.1 also uses the ReAct-v3 ablation to show that the bulk of Researcher's aggregate advantage over ReAct-v1 and ReAct-v2 comes from the choice of model family rather than from the planning stage itself.

Separately from the four browsing-enabled agents above, Section 5.3.3 reports a Retrieval configuration that serves as a tool-ablation reference: it shares the same GPT-4o-mini backbone and index but is restricted to a single `search_{email,chat,meeting,file}` call per case and cannot invoke `read_file` (full-body reader) or `search_in_file` (exact-term scan). It is not included in the main results table; its purpose is solely to isolate the contribution of full-content browsing on top of snippet retrieval.

5.1.2 Evaluation Metrics. Our experiments instantiate the configurable harness (Section 3.5) with GPT-4o as the judge and three judge prompts that together score correctness, retrieval quality, and citation grounding. The resulting metric configuration is:

- **Assertion score (Assr.)** Each gold assertion a_i is presented to the judge individually; the judge returns True/False plus a brief rationale. The case score is Assr. $= |\{a_i : \text{True}\}|/|A|$.
- **Tool-search relevance (TS-Rel)** A single judge call evaluates (i) whether the selected tools were appropriate for the query, (ii) whether the retrieved evidence was relevant, and (iii) whether the response is grounded in the tool results rather than hallucinated. Returns a holistic score in $[0, 1]$.
- **Response-citation quality (RC-Rel)** A separate judge call checks (i) whether a `## References` section exists, (ii) whether each cited artifact ID appears in the tool-call trace (grounded) or was fabricated (hallucinated), and (iii) whether inline citation markers appear in the response body. Returns a score in $[0, 1]$.
- **Combined citation score (Cite)** Equal-weight mean of TS-Rel and RC-Rel, summarising end-to-end evidence handling.

Deterministic diagnostics collected without LLM calls: TS-n (retrieved-item count) and RC-n (response citation count).

Pass/fail rule. Following the two-gate criterion motivated in Section 3.5, a case is marked *passed* when Assr. ≥ 0.75 *and* RC-Rel ≥ 0.7, i.e., both the correctness gate and the grounding gate hold simultaneously. Pass rate is the fraction of cases that satisfy both gates. We verified rank stability under this threshold choice by sweeping the assertion gate $\tau \in \{0.70, 0.75, 0.80, 0.90, 1.00\}$ while holding the RC-Rel gate fixed

at 0.7. The agent ranking is preserved in every (agent, tenant) cell (Appendix Table 22): no cell reverses ordering across τ, and pass-rate vectors are monotone non-increasing in τ as expected.

5.2 Main Results

5.2.1 Overall Results. Table 18 reports the decomposed metrics across all twelve agent–tenant combinations, with each pass-rate estimate augmented by a 95% Wilson confidence interval. Paired Wilcoxon signed-rank tests between selected agent pairs on matched case IDs (reported inline below) are summarised in Appendix Table 21 for completeness.

Table 18. Per-tenant and per-agent evaluation results. *Pass*: pass rate (with 95% Wilson CI in brackets). *Assr.*: assertion score. *TS-Rel*: tool-search relevance. *TS-n*: mean items retrieved. *RC-Rel*: response-citation relevance. *RC-n*: mean grounded citations. *Cite*: composite citation score (mean of TS-Rel and RC-Rel). Bold marks the best per-tenant value.

Tenant	Agent	Cases	Pass [95% CI]	Assr.	TS-Rel	TS-n	RC-Rel	Cite
Bertrand	ReAct-v1	162	0.414 [.341,.491]	**0.736**	**0.803**	19.2	0.593	0.698
	ReAct-v2	162	0.259 [.198,.332]	0.729	0.650	15.8	0.479	0.565
	ReAct-v3	162	0.395 [.323,.472]	0.666	0.789	43.0	**0.665**	**0.727**
	Researcher	162	**0.432** [.358,.509]	0.625	0.762	**61.1**	0.640	0.701
Cambford	ReAct-v1	162	0.228 [.171,.299]	0.515	0.717	18.3	0.449	0.583
	ReAct-v2	162	0.222 [.165,.292]	0.489	0.701	17.0	0.440	0.571
	ReAct-v3	162	0.296 [.231,.371]	**0.539**	**0.808**	36.4	**0.700**	**0.754**
	Researcher	162	**0.346** [.277,.422]	0.543	0.793	**61.1**	0.684	0.738
ZAI	ReAct-v1	201	0.269 [.212,.334]	0.560	0.797	17.9	0.558	0.677
	ReAct-v2	201	0.154 [.111,.211]	0.473	0.717	13.5	0.493	0.605
	ReAct-v3	201	**0.363** [.300,.432]	**0.569**	**0.797**	36.6	**0.740**	**0.768**
	Researcher	201	0.289 [.230,.355]	0.503	0.765	**62.5**	0.623	0.694

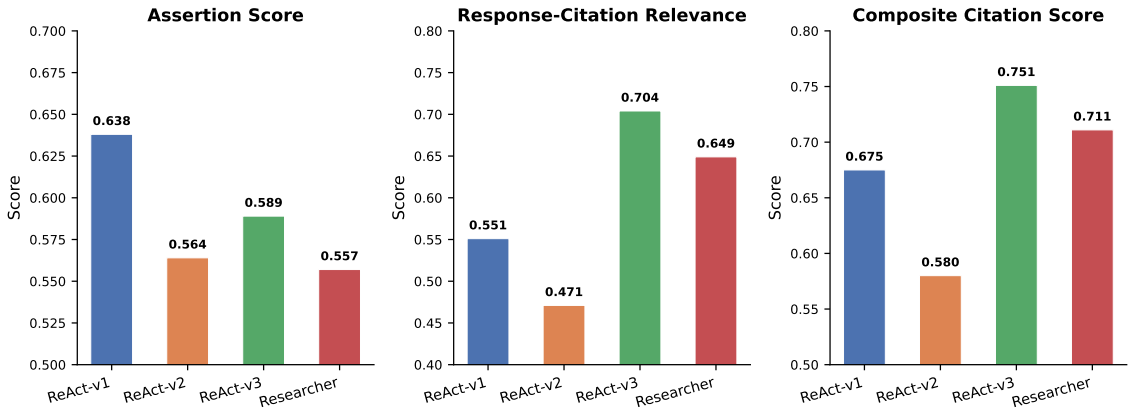

Fig. 11. Weighted-aggregate metrics across all tenants. *Left*: assertion score—ReAct-v1 leads (0.638). *Middle*: response-citation relevance—ReAct-v3 leads (0.704). *Right*: composite citation score (mean of TS-Rel and RC-Rel)—ReAct-v3 leads (0.751).

5.2.2 Comparisons of the Four Methods. The aggregate picture changes substantially once the model-size variable is held fixed. Researcher (0.355 pass, 95% CI [.316,.395]) and the ReAct-v3 ablation (0.352 pass, 95% CI [.312,.394]) are statistically indistinguishable in aggregate, even though ReAct-v3 uses no planning stage and less than a third of the prompt tokens (27.4k vs. 92.3k). ReAct-v3 also achieves the highest composite citation score (Cite = 0.751) and tool-search relevance (TS-Rel = 0.798), indicating that its retrieval quality exceeds all other agents despite using fewer tool calls (1.23 vs. 4.85 for Researcher). ReAct-v1 (GPT-4o + verbose prompt) trails at 0.323 pass but attains the best mean assertion score (0.638), while ReAct-v2 is clearly dominated at 0.212 across all metrics. The per-tenant picture is more nuanced: Researcher remains best on Bertrand (0.432) and Cambford (0.346, and the only cell where the advantage over ReAct-v1 is significant at p=5.6e−3), but ReAct-v3 is the single best agent on ZAI (0.363 pass, 0.740 RC-Rel, 0.768 Cite) and significantly beats Researcher there (p=0.032, paired Wilcoxon). ReAct-v3 also outperforms ReAct-v2 at fixed model on ZAI (p=5.7e−8), isolating the benefit of the verbose prompt. These contrasts reshape the headline conclusion: rather than "plan-and-execute wins", the evidence supports "plan-and-execute wins on temporally dispersed, multi-hop workloads (Cambford) but ties or loses on dense, bursty workloads (ZAI, Bertrand) once the model-family confound is removed."

5.3 Ablation Studies

This section isolates the contribution of each design axis along which our agent configurations differ. We consider three orthogonal axes in turn: architecture (ReAct vs. plan-and-execute at fixed model), backbone model (GPT-4o vs. GPT-4o-mini at fixed architecture and prompt), and tool surface (snippet-only retrieval vs. full-content browsing).

5.3.1 ReAct vs. Plan-and-Execute. The primary architectural difference between Researcher and the two reactive agents is the separation of planning from execution described in Section 5.1.1. Because Researcher and ReAct-v3 share the GPT-4o-mini backbone and differ only in the presence of a planning stage, the ReAct-v3 ↔ Researcher contrast isolates the planning effect at fixed model. The planning-stage design directly explains three empirical patterns.

First, Researcher retrieves 61.6 items per case versus 15–19 for the GPT-4o reactive agents (ReAct-v3 retrieves 38.5)—a factor of ∼3–4×. The planning prompt explicitly requires correlated search across emails, chats, meetings, and files, whereas the reactive agents only pursue additional artifact types opportunistically. This broader retrieval footprint is the primary driver of the response-citation advantage: with more retrieved artifacts available, the execution model has more material to cite, raising the grounded-citation count from 1.87 (v1) to 2.81 (Researcher).

Second, the planning stage is most beneficial when queries require temporal multi-hop reasoning. On Cambford, where cases frequently require reconstructing policy evolution across weeks of committee emails and subsequent meeting threads, Researcher improves pass rate by 52% relative to ReAct-v1 (0.346 vs. 0.228, paired Wilcoxon p=5.6e−3) and achieves the highest citation score in any cell (0.738). On Bertrand, where the smaller, denser organization makes most answers reachable via a single well-formed search, the pass-rate gap shrinks to 4% (0.432 vs. 0.414) and is no longer significant (p=0.68). On ZAI, where evidence is concentrated in dense operational bursts, ReAct-v1 closes to within 2 percentage points of Researcher on pass rate (0.269 vs. 0.289, p=0.57) and substantially exceeds it on assertion score (0.560 vs. 0.503).

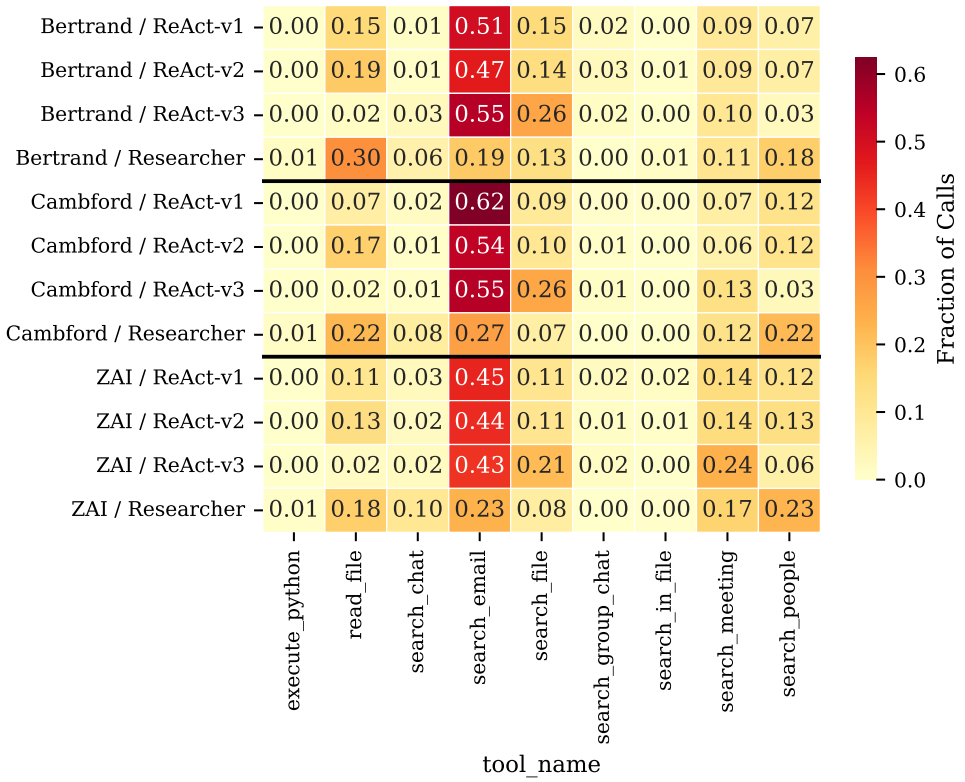

Fig. 12. Tool-call distribution by agent and tenant. Rows are grouped by tenant; columns show individual tools. Researcher allocates calls across more tool types in every tenant, while the reactive agents concentrate on `search_messages` and `search_people`. Cross-tenant variation is visible: ZAI's larger corpus drives higher `read_file` usage across all agents.

Third, despite leading on pass rate and citation, Researcher lags on assertion score (0.557 vs. 0.638 for v1). The smaller GPT-4o-mini model has lower per-call reasoning capacity than GPT-4o. The planning stage further compounds this by guiding execution toward broad retrieval at the expense of precise fact synthesis. The result is a coverage-vs.-precision trade-off: Researcher achieves broad retrieval coverage at the cost of per-assertion accuracy.

5.3.2 Model-wise Comparison (GPT-4o vs. GPT-4o-mini). The ReAct-v1 ↔ ReAct-v3 pair holds architecture and the verbose prompt fixed while swapping the backbone from GPT-4o to GPT-4o-mini, cleanly isolating the model-family effect. Combined with the planning contrast above, this lets us decompose the aggregate Researcher − ReAct-v1 gap into two orthogonal contributions. Holding architecture and prompt fixed and varying only model family yields per-tenant pass-rate changes of −0.019 / +0.068 / +0.094 on Bertrand / Cambford / ZAI respectively, i.e., GPT-4o-mini is neutral-to-better on all three tenants and strongly better on the bursty ZAI workload. Holding model fixed and adding the planning stage (ReAct-v3 → Researcher) yields +0.037 / +0.050 / −0.074: planning gives a modest boost on Bertrand and Cambford but is actively harmful on ZAI, where the planner issues extra retrieval steps that

consume context budget without adding new evidence. The planner's aggregate $0.355 > 0.323$ advantage over ReAct-v1 is therefore a composite of a real planning benefit on Cambford and a model-family effect that moves in the opposite direction on ZAI. A practitioner choosing an agent should read Table 18 row-by-row rather than relying on the aggregate, because the correct winner depends strongly on the target tenant's temporal structure.

5.3.3 Tool-wise Ablation: Full-Content Browsing. The agents reported above are all granted a rich browsing surface—in addition to the four `search_{email,chat,meeting,file}` endpoints that return ranked snippets, each of them can call `read_file` to fetch the complete body of an already-identified artifact and `search_in_file` to perform an exact-term scan inside that body. To quantify what this full-content browsing buys, we run a tool ablation in which every other component of the runtime is held fixed and the tool surface is collapsed to snippet retrieval only. The resulting Retrieval configuration is defined in Section 5.1.1: it issues exactly one `search_*` call per case against the same hybrid index, produces a grounded answer from the returned snippets, and is forbidden from invoking either `read_file` or `search_in_file`. The gap between Retrieval and the reactive agents therefore isolates the joint contribution of (i) iterating beyond a single retrieval, (ii) reading the full artifact body instead of a snippet, and (iii) using exact-term matches inside that body to disambiguate candidates.

Table 19. Mean tool calls per case by agent, aggregated across all three tenants. *Email/Chat/Meeting/File*: content-search endpoints returning ranked snippets. *People*: name-to-ID resolution. *Read*: full-body artifact retrieval (`read_file`). *Search-in*: exact-term scan inside an artifact (`search_in_file`). Retrieval is restricted to a single snippet-search call by construction.

Agent	Email	Chat	Meet.	File	People	Read	Search-in	Total
ReAct-v1	0.81	0.05	0.17	0.19	0.18	0.27	0.01	1.69
ReAct-v2	0.81	0.06	0.17	0.18	0.16	0.17	0.01	1.56
ReAct-v3	0.62	0.04	0.20	0.30	0.05	0.02	0.00	1.23
Researcher	0.96	0.34	0.56	0.39	**0.87**	**0.97**	0.03	**4.17**
Retrieval	0.96	0.01	0.03	—	—	—	—	1.00

Table 19 and Figure 13 contrast tool-usage patterns and the browsing-ablation results. Three observations stand out. (1) *Tool diversity separates the planner from the reactive agents.* Researcher calls `search_people` (0.87/case) and `read_file` (0.97/case) far more than any reactive agent; its planning stage explicitly mandates name-to-ID resolution before content search. The reactive agents concentrate on `search_email` and rarely invoke full-body reading. (2) *Snippet-only retrieval is a genuine floor, not a ceiling.* Pooled over 525 matched cases, Retrieval attains 24.2% pass versus 35.5% for Researcher and 35.2% for ReAct-v3—an 11–12 point absolute gap. The gap maps onto exactly the capability that `read_file` and `search_in_file` unlock: retrieving a snippet suffices to cite an artifact, but reading its full body is what turns a plausible citation into a correct assertion. (3) *Browsing does not help a bad agent.* ReAct-v2, whose prompt does not exploit the browsing tools effectively, is statistically indistinguishable from Retrieval on all three tenants ($p \in \{0.35, 1.00, 0.12\}$) and even trails it on ZAI (15.4% vs. 20.9%). In other words, access to full-content browsing is necessary but not sufficient; the agent must actually use it.

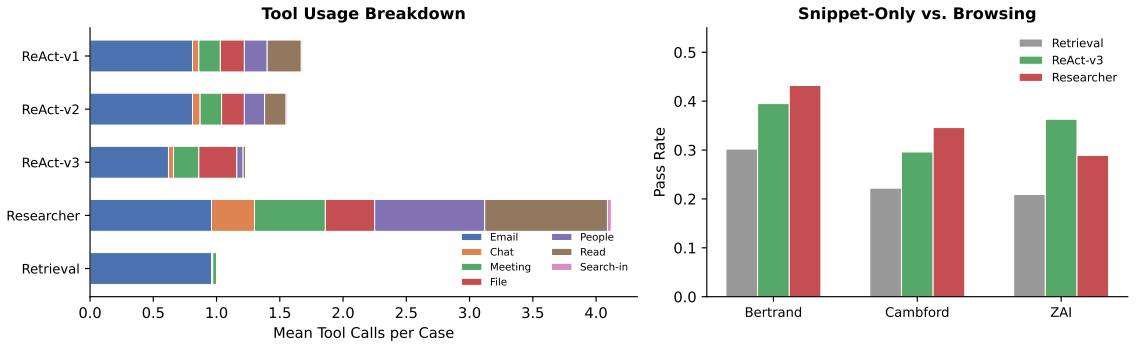

Fig. 13. *Left*: stacked tool-call breakdown per agent (mean calls/case across 525 cases). Researcher uses 4× more tools than the reactive agents, dominated by `search_people` and `read_file`. Retrieval is restricted to a single snippet-search call. *Right*: per-tenant pass-rate comparison between snippet-only Retrieval and two browsing-enabled agents. Browsing yields a consistent 9–15 point lift on Bertrand and Cambford; on ZAI, ReAct-v3 gains 15 points while Researcher gains only 8.

5.4 Case Studies

5.4.1 Effect of Prompt Engineering. ReAct-v1 and ReAct-v2 share the same architecture, backbone model (GPT-4o), and tool set, yet their aggregate pass rates differ by 11 percentage points (32.3% vs. 21.2%). System-prompt verbosity is the primary driver of this gap. ReAct-v1's eight fully worked examples concretely demonstrate each behavioral pattern: the agent sees a complete Thought→Tool→Observation chain for dual-term person lookup, pronoun resolution, multi-query decomposition, Python analytics, and citation formatting. ReAct-v2's five bullet-style examples state the same requirements but as abstract directives rather than worked instances. The consequence is most visible in citation behavior: v2's response-citation relevance (0.471) is 8 points below v1's (0.551), suggesting that the model benefits more from seeing the correct citation format demonstrated than from being told to produce it. Tool-search relevance drops even more sharply (0.689 vs. 0.785), indicating that the condensed prompt also impairs the quality of the search queries themselves, not just the final formatting. The composite citation score reflects both gaps: v1's aggregate Cite (0.675) exceeds v2's (0.580) by nearly 10 points.

5.4.2 Scenario-Level Analysis. The three tenants stress different agent capabilities, and the per-tenant results expose the conditions under which each architecture excels or fails.

Bertrand & Co. (Legal). Bertrand is the smallest and densest tenant (Table 14), so most entities are reachable within two hops and a single well-executed tool call frequently suffices. All four agents perform comparably on pass rate—ReAct-v1 at 0.414, ReAct-v3 at 0.395, and Researcher at 0.432—and ReAct-v1 leads decisively on assertion score (0.736 vs. 0.625 for Researcher). ReAct-v3 achieves the highest RC-Rel (0.665) and Cite (0.727). The planning overhead provides near-zero benefit when evidence is organizationally concentrated.

Univ. of Cambford (Education). This tenant, designed around multi-week committee processes (Section 4), is the most challenging for reactive agents: without an explicit dependency-chain plan, ReAct-v1 achieves

only 0.228 and ReAct-v2 0.222—barely distinguishable. Researcher's planning stage raises pass rate to 0.346 and Cite to 0.738, the highest across all twelve cells. ReAct-v3 also benefits from the GPT-4o-mini backbone here, reaching 0.296 pass and the highest RC-Rel (0.700) and Cite (0.754) on this tenant. This scenario is the clearest demonstration of when plan-and-execute provides a qualitative advantage over reactive reasoning.

ZAI Intelligence (Software / AI).. ZAI combines the largest scale (Table 14) with the highest chat burstiness ($B = 0.658$, Table 15), producing dense same-day clusters of engineering activity. Here the reactive agents are competitive because a well-targeted single search often retrieves several related artifacts simultaneously. ReAct-v3 is the outright winner on ZAI (0.363 pass, 0.740 RC-Rel, 0.768 Cite), while ReAct-v1 achieves 0.269 pass and the highest per-tenant assertion score (0.560)—substantially above Researcher's 0.503. Researcher's 0.289 pass rate places it below both ReAct-v3 and ReAct-v1, suggesting that the planning overhead can be counterproductive when evidence is dense: the planner introduces additional, potentially irrelevant search steps that consume context budget without improving accuracy.

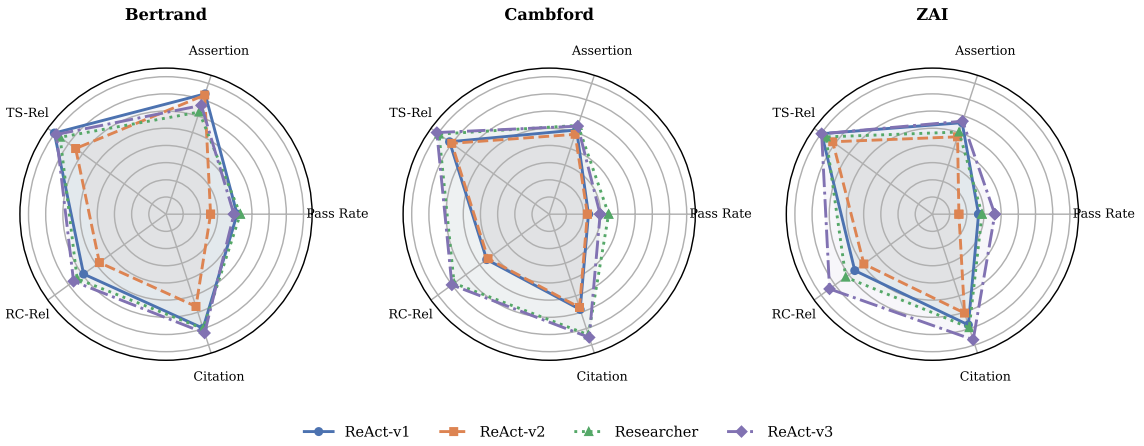

Fig. 14. Per-tenant agent profiles. Each subplot shows the four agents on a shared radar for one tenant. On Bertrand (dense, small) all agents overlap; on Cambford (temporal multi-hop) Researcher's citation and pass-rate advantage widens; on ZAI (large, bursty) ReAct-v3 matches or exceeds the planner on assertion accuracy.

5.4.3 Query-Type Analysis. The evaluation set comprises the three query categories defined in Section 3.4, generated with a 40/40/20 target split. Breaking down assertion and citation metrics by query type reveals how each architecture handles increasing retrieval complexity. Query types are assigned by the generator at case-creation time and recorded in the `query_type` field of the evaluation JSON; the reported split is derived from that field rather than from any post-hoc classification.

On targeted artifact search queries, which require locating a single artifact, all three agents achieve their highest assertion scores and the inter-agent gap is narrow. This confirms that when evidence is concentrated in a single entity, even a minimal reactive loop suffices. On multi-hop reasoning queries, assertion scores drop across the board as agents must connect information across multiple artifacts; here ReAct-v1's detailed

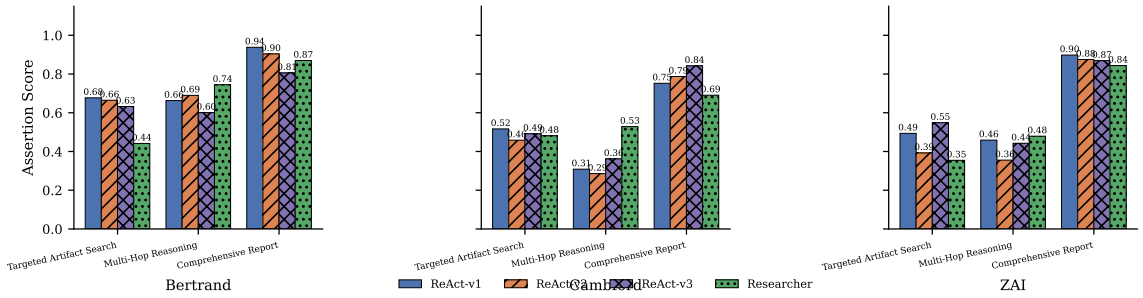

Fig. 15. Mean assertion score by query type and tenant. All agents perform best on targeted artifact search queries. The gap between agents widens on multi-hop reasoning and comprehensive report queries, where ReAct-v1's precision advantage is most pronounced.

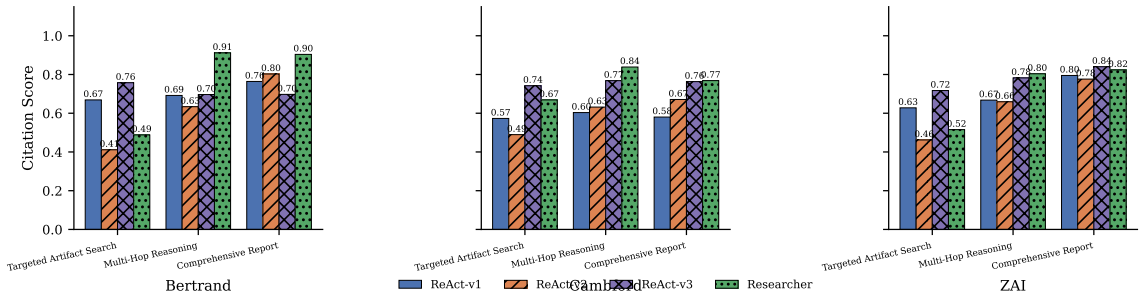

Fig. 16. Mean citation score by query type and tenant. Researcher leads on comprehensive report queries where its planning stage systematically covers multiple source types, while the reactive agents are competitive on targeted artifact search queries.

worked examples help it maintain higher per-assertion accuracy than Researcher, whose broader retrieval introduces noise. On comprehensive report queries, Researcher's planning stage provides the clearest advantage in citation quality: its dependency-chain decomposition ensures that multiple source types are systematically retrieved and cited, while the reactive agents struggle to organize comprehensive responses from ad-hoc retrieval sequences.

Post-hoc distribution audit. To sanity-check that the generator-declared category labels reflect the actual surface form of the queries, we ran a keyword-based audit on all 380 assertions from the ReAct-v1 × Bertrand run. The observed distribution (∼31% email-anchored, ∼20% file-anchored, ∼21% explicitly cross-source, ∼5% meeting, ∼1% chat, remainder untagged) approximately matches the generator-declared split and confirms that cross-source queries are well represented. This is an automated lexical audit, not a human validation study; a full inter-annotator reliability study on the gold assertions is an open task left for future work.

5.4.4 Cost–Effectiveness Trade-off. The results reveal a clear cost-effectiveness frontier. Researcher achieves the highest pass rate (35.5%) and citation quality (0.711) but consumes 5.2× more prompt tokens (92,280 vs. 17,778) than ReAct-v1. For deployments where retrieval completeness and citation traceability are the primary objectives—legal research, compliance auditing, or due-diligence workflows—this overhead may

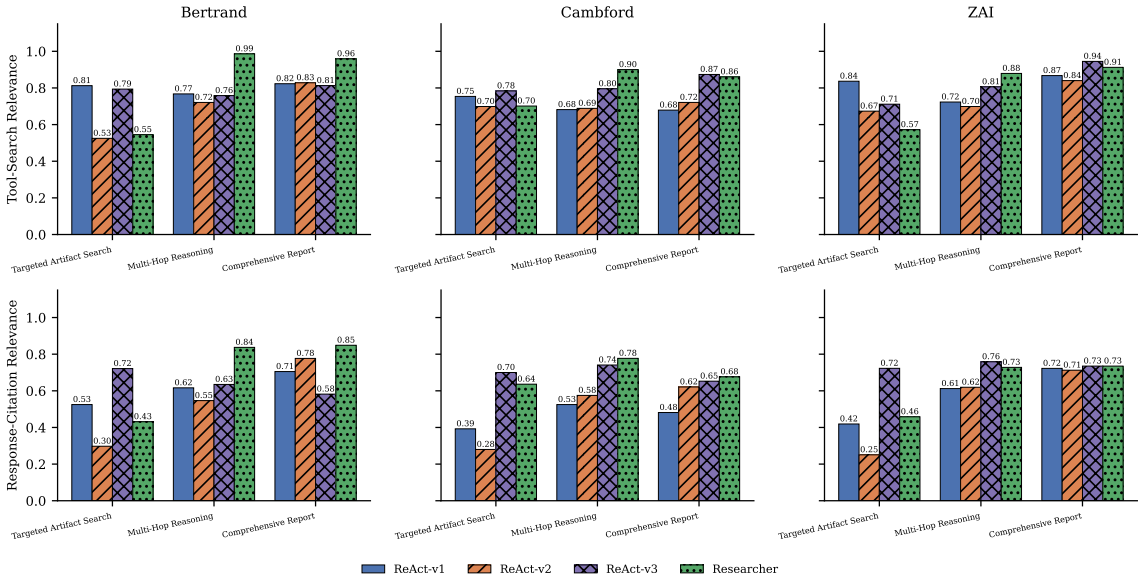

Fig. 17. Citation decomposition (tool-search relevance and response-citation relevance) by query type. Top row: tool-search relevance; bottom row: response-citation relevance. The performance gap between agents is most visible on comprehensive report queries, where Researcher's broader retrieval strategy produces higher relevance across both sub-metrics.

be justified. For token-sensitive applications or scenarios with dense, recent evidence, ReAct-v1 provides 32.3% pass rate at roughly one-fifth the prompt-token cost with higher factual precision (assertion 0.638 vs. 0.557).

ReAct-v2 does not occupy a Pareto-optimal point on this frontier: it matches ReAct-v1 on tool-call count (1.70) but lags on all quality metrics. This finding suggests that prompt compression is costly in the absence of compensating architectural changes—a concise prompt is not equivalent to an equally effective one. The ReAct-v3 ablation reshapes this frontier: at 0.352 aggregate pass rate, 0.751 composite citation, and \$1.14–\$1.19 per 100 cases, it Pareto-dominates ReAct-v1 (same architecture, larger model) and Researcher (same model, added planning) on dollar cost while matching their quality. The practical implication is that a verbose, worked-example prompt on a small model is the most cost-effective configuration we measured.

Dollar-cost breakdown. Table 20 converts the logged per-case prompt and completion token counts to Azure list prices (GPT-4o: \$2.50 / \$10.00 per 1M input / output tokens; GPT-4o-mini: \$0.15 / \$0.60 per 1M). The judge column is the amortised per-case cost of the four judge calls (per-assertion correctness check, tool-search relevance, response-citation quality, and side-by-side comparison) under the same list prices, averaged over the per-tenant case count. The resulting frontier is dominated by ReAct-v3: it matches Researcher's pass rate at \$1.92–\$2.30 per 162–201-case tenant run versus \$3.15–\$3.91 for Researcher, and at roughly one-fifth the cost of ReAct-v1. ReAct-v2 remains Pareto-dominated: it is cheaper than ReAct-v1 in dollar terms but delivers materially lower pass rates.

Table 20. Dollar-cost breakdown per tenant run, derived from logged token counts at Azure list prices. *Agent $*: cost of the agent loop. *Judge $*: amortised cost of the four GPT-4o judge calls. *Total $*: sum. *$/100*: cost per 100 evaluation cases.

Tenant	Agent	Agent $	Judge $	Total $	$/100
Bertrand	ReAct-v1	9.50	1.16	10.66	6.58
	ReAct-v2	8.00	1.17	9.17	5.66
	ReAct-v3	0.78	1.14	1.92	1.19
	Researcher	1.99	1.16	3.15	1.94
Cambford	ReAct-v1	7.97	1.27	9.24	5.70
	ReAct-v2	7.64	1.26	8.90	5.49
	ReAct-v3	0.61	1.26	1.87	1.15
	Researcher	1.95	1.26	3.21	1.98
ZAI	ReAct-v1	11.24	1.37	12.61	6.27
	ReAct-v2	9.12	1.38	10.50	5.22
	ReAct-v3	0.93	1.37	2.30	1.14
	Researcher	2.53	1.38	3.91	1.95

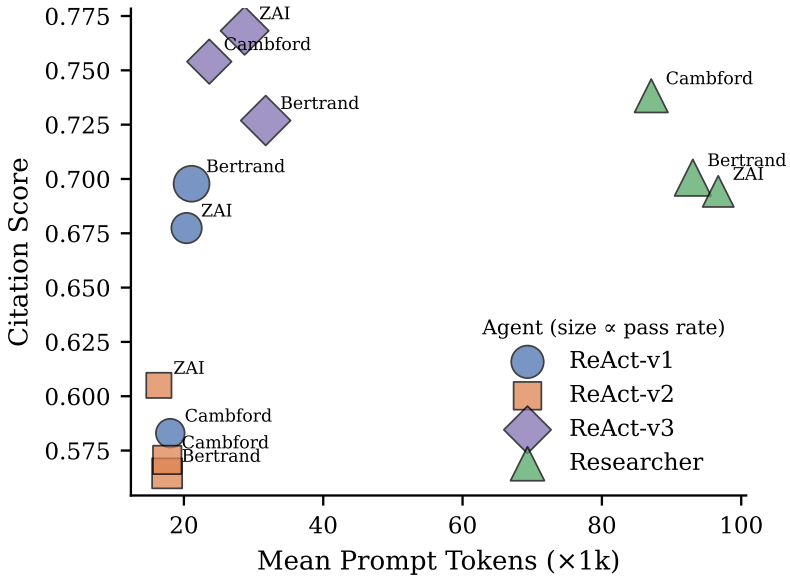

Fig. 18. Cost–quality frontier: mean prompt tokens vs. citation score. Marker size is proportional to pass rate. Researcher achieves the highest citation quality at 5.2× the token cost of the reactive agents; ReAct-v2 does not occupy a Pareto-optimal point.

5.5 Threats to Validity

Single-judge dependence. All reported quality metrics are produced by a GPT-4o judge. To test robustness against judge-model choice, we attempted a replication with GLM-4.7-FlashX as an alternative judge; however, the GLM response formatting diverged sufficiently from our parser's expected schema that the automatically extracted pass rates were systematically deflated to 1–2% across all agents, which we attribute to parse failures rather than to model disagreement. We therefore do not report cross-model judge

numbers, and flag cross-judge robustness as an open validation item. The agent-level rankings reported here should be interpreted as conditional on a single-judge (GPT-4o) protocol.

5.6 Cross-Model Evaluation and Bias Mitigation

A synthetic benchmark must guard against circularity: if the same model family generates the data, runs the agent, and judges the output, self-preference bias can inflate scores. The EABench protocol treats the generator, runtime, and judge as three independent factors. In the experiments reported above, the generator and judge both use GPT-4o while the runtime varies across four agent configurations that differ in architecture, prompt design, and model size (GPT-4o for ReAct-v1 and ReAct-v2; GPT-4o-mini for ReAct-v3 and the Researcher). This setup reflects the realistic practitioner design space in which architecture, prompting, and model choice are varied jointly. To fully control for generator–judge bias, the framework supports a cross-model matrix in which tenants generated by model family G_i are evaluated by runtime R_j and judged by J_k, with the restriction that no conclusion is drawn from a single diagonal cell. Diagonal effects—where a model consistently scores higher only when generating or judging its own data—can then be identified as self-preference rather than genuine method improvement.

A cross-judge replication with GLM-4.7-FlashX as an alternative judge produced pass rates of 1–2% across all agents, which we traced to parse-level schema mismatches rather than genuine disagreement between judges. We therefore treat cross-judge robustness as an open validation task (Section 5.5) and rely on the generator–runtime separation—with the runtime varying across four agent configurations (three GPT-4o / GPT-4o-mini ReAct variants plus the plan-and-execute Researcher)—to bound self-preference effects. Importantly, at fixed judge, the ReAct-v3 ablation (GPT-4o-mini + verbose ReAct prompt) ties Researcher in aggregate pass rate and beats it on ZAI; this model-family-matched comparison cannot be explained by GPT-4o judge preference for GPT-4o-mini runtimes, because the comparison is within the GPT-4o-mini family. The pattern we report therefore reflects an architecture-and-prompt effect rather than a generator–judge artefact.

5.7 Discussion

The experiments expose three principal findings. First, the benefit of plan-and-execute over reactive reasoning is scenario-specific rather than universal: only on Cambford (multi-week committee workflows) does Researcher significantly outperform ReAct-v1 (p=5.6e−3), while on ZAI (dense, bursty engineering activity) a controlled ReAct-v3 ablation at the same model family significantly beats Researcher (p=0.032). The aggregate ordering that appears to favour Researcher is therefore a composition of a genuine planning benefit on one scenario and a model-family contrast on the others—a decomposition that a monolithic, single-scenario benchmark could not surface. Second, prompt engineering matters as much as architecture: the 11-point pass-rate gap between ReAct-v1 and ReAct-v2—identical architectures differing only in prompt verbosity—exceeds the 3-point gap between ReAct-v1 and Researcher on two of three tenants, and the ReAct-v3 ablation further shows that a verbose prompt on a small model (0.352 aggregate pass, $1.14/100 cases) Pareto-dominates both the larger-model reactive and planning alternatives on cost. Third, the decomposed citation metrics reveal that retrieval quality and citation formatting are largely independent failure modes: ReAct-v1 leads on tool-search relevance (0.799) while Researcher

leads on response-citation quality (0.649), indicating that broad retrieval and well-structured references require different capabilities.

These findings suggest that future work should explore hybrid architectures that conditionally activate planning based on query complexity, as well as prompt-optimization techniques that maintain the behavioral specificity of verbose prompts at lower token cost.

6 Conclusion and Discussion

EABench reframes enterprise-agent benchmarking as an end-to-end systems problem. Instead of providing only a fixed dataset or only a runtime abstraction, it joins four ingredients in a single platform: coherent tenant generation, configurable agent execution, configurable query and judge generation, and scenario-aware debugging under access control. This combination is what makes the benchmark useful for research rather than merely descriptive. It lets investigators ask not only whether one agent outperforms another, but why that difference appears and whether it survives changes in scenario, model family, retrieval stack, or judge configuration.

The framework still has clear limitations. It currently assumes English-language enterprise communication, depends on sufficiently capable foundation models to preserve long-range narrative coherence during generation, and relies on judge prompts whose scope must be reviewed carefully when researchers introduce new task families. In addition, interactive debugging is a complement to formal evaluation rather than a substitute for it: visual inspection can diagnose failure modes, but final comparisons must still be grounded in reproducible case-level metrics.

These limitations point directly to future work. The next priorities are broader multilingual support, richer sandbox backends for code-centric enterprise tasks, stronger controls against generator–judge self-preference bias, and expanded human validation protocols for both realism and scoring quality. Even in its current form, however, EABench provides a practical and extensible foundation for studying enterprise LLM agents under realistic data, realistic constraints, and experimentally meaningful variation.

References

[1] Amirhossein Abaskohi, Tianyi Chen, Miguel Munoz-Marmol, Curtis Fox, Amrutha Varshini Ramesh, Etienne Marcotte, Xing Han Lu, et al. 2025. DRBench: A Realistic Benchmark for Enterprise Deep Research. *arXiv preprint arXiv:2510.00172* (2025).

[2] Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios N. Angelopoulos, Tianyi Li, Dacheng Li, et al. 2024. Chatbot Arena: An open platform for evaluating LLMs by human preference. *arXiv preprint arXiv:2403.04132* (2024).

[3] Prafulla Kumar Choubey, Xiangyu Peng, Shilpa Bhagavath, Kung-Hsiang Huang, Caiming Xiong, and Chien-Sheng Wu. 2025. Benchmarking Deep Search over Heterogeneous Enterprise Data. *arXiv preprint arXiv:2506.23139* (2025).

[4] Open-Source Community. 2025. GAIA-2: The Next Generation of General AI Assistants Benchmark. arXiv:2501.00000 [cs.AI]

[5] A. Drouin, M. Gasse, M. Caccia, I. H. Laradji, M. Del Verme, T. Marty, et al. 2024. WorkArena: How capable are web agents at solving common knowledge work tasks? *arXiv preprint arXiv:2403.07718* (2024).

[6] E. Glazer, E. Erdil, T. Besiroglu, D. Chicharro, E. Chen, A. Gunning, et al. 2024. FrontierMath: A benchmark for evaluating advanced mathematical reasoning in AI. *arXiv preprint arXiv:2411.04872* (2024).

[7] Kwang-Il Goh and Albert-László Barabási. 2008. Burstiness and memory in complex systems. *EPL (Europhysics Letters)* 81, 4 (2008), 48002. doi:10.1209/0295-5075/81/48002

[8] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *ICLR*.

[9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation

for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
[10] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, et al. 2023. AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688* (2023).
[11] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. GAIA: A Benchmark for General AI Assistants. In *ICLR*.
[12] Microsoft. 2023. Microsoft 365 Copilot: The AI-powered productivity tool for work. Microsoft Blog.
[13] A. Mitra, L. Del Corro, S. Mahajan, A. Codas, C. Simoes, S. Agrawal, et al. 2024. AgentInstruct: Toward generative teaching with agentic flows. *arXiv preprint arXiv:2407.03502* (2024).
[14] Krista Opsahl-Ong, Arnav Singhvi, Jasmine Collins, Ivan Zhou, Cindy Wang, Ashutosh Baheti, Owen Oertell, Jacob Portes, Sam Havens, Erich Elsen, Michael Bendersky, Matei Zaharia, and Xing Chen. 2026. OfficeQA Pro: An Enterprise Benchmark for End-to-End Grounded Reasoning. *arXiv preprint arXiv:2603.08655* (2026).
[15] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, et al. 2023. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789* (2023).
[16] Divyanshu Saxena, Rishikesh Maurya, Xiaoxuan Ou, Gagan Somashekar, Shachee Mishra Gupta, Arun Iyer, Yu Kang, Chetan Bansal, Aditya Akella, and Saravan Rajmohan. 2025. Continuous Benchmark Generation for Evaluating Enterprise-Scale LLM Agents. *arXiv preprint arXiv:2511.10049* (2025).
[17] Harsh Vishwakarma, Ankush Agarwal, Ojas Patil, Chaitanya Devaguptapu, and Mahesh Chandran. 2025. Can LLMs Help You at Work? A Sandbox for Evaluating LLM Agents in Enterprise Environments. In *EMNLP*.
[18] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim. 2023. Plan-and-Solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *ACL*.
[19] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, et al. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems (NeurIPS)*.
[20] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *EMNLP*.
[21] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. 2024. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045* (2024).
[22] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. 2022. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629* (2022).
[23] W. Zhao, X. Ren, J. Hessel, C. Cardie, Y. Choi, and Y. Deng. 2024. WildChat: 1M ChatGPT interaction logs in the wild. In *ICLR*.
[24] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, et al. 2023. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *NeurIPS*.
[25] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, S. Sridhar, et al. 2023. WebArena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854* (2023).

A Extended Experimental Results

This appendix collects supporting tables that accompany the main results in Section 5. Table 21 reports the full pairwise Wilcoxon signed-rank test matrix on paired cases (used in Section 5 to substantiate that architectural gaps are statistically significant on some tenants but not others). Table 22 reports pass-rate stability as the assertion-match threshold τ is varied from 0.70 to 1.00, supporting the robustness claim in the Evaluation Metrics subsection.

B Agent Configurations and Prompts

This appendix reproduces the full system prompts and configurations used for each of the five agent variants evaluated in Section 5. Table 23 summarises the key configuration parameters; the subsections that follow give the verbatim prompts.

All browsing-enabled agents (ReAct-v1/v2/v3 and Researcher) share ten tools: `search_email`, `search_chat`, `search_group_chat`, `search_channel`, `search_meeting`, `search_file`, `read_file`, `search_in_file`, `search_people`, and `execute_python`. The Retrieval baseline is restricted to four search tools (`search_email`,

Table 21. Pairwise Wilcoxon signed-rank tests on matched cases, per tenant, for the four browsing-enabled agents. n is the number of paired cases; *Pass A* and *Pass B* are the per-agent pass rates on the matched subset; p is the two-sided Wilcoxon signed-rank p-value. The retrieval-only RETRIEVAL ablation is reported separately in Section 5.3.3 and is not repeated here. Boldface marks $p < 0.05$.

Agent A	Agent B	Tenant	n	Pass A / B	p
react_v1	react_v2	Cambford	162	0.228 / 0.222	0.847
react_v1	react_v2	ZAI	201	0.269 / 0.154	**1.0e-4**
react_v1	react_v2	Bertrand	162	0.414 / 0.259	**2.7e-4**
react_v1	researcher	Cambford	162	0.228 / 0.346	**5.6e-3**
react_v1	researcher	ZAI	201	0.269 / 0.289	0.572
react_v1	researcher	Bertrand	162	0.414 / 0.432	0.680
react_v1	react_v3	Cambford	162	0.228 / 0.296	0.101
react_v1	react_v3	ZAI	201	0.269 / 0.363	**1.8e-3**
react_v1	react_v3	Bertrand	162	0.414 / 0.395	0.668
react_v2	researcher	Cambford	162	0.222 / 0.346	**1.2e-3**
react_v2	researcher	ZAI	201	0.154 / 0.289	**5.7e-5**
react_v2	researcher	Bertrand	162	0.259 / 0.432	**7.5e-5**
react_v2	react_v3	Cambford	162	0.222 / 0.296	**0.034**
react_v2	react_v3	ZAI	201	0.154 / 0.363	**<1e-4**
react_v2	react_v3	Bertrand	162	0.259 / 0.395	**4.5e-3**
researcher	react_v3	Cambford	162	0.346 / 0.296	0.182
researcher	react_v3	ZAI	201	0.289 / 0.363	**0.032**
researcher	react_v3	Bertrand	162	0.432 / 0.395	0.405

Table 22. Pass-rate sensitivity to the assertion-match threshold τ. Response-citation gate held at 0.70. Rows are agent–tenant cells; columns are $\tau \in \{0.70, 0.75, 0.80, 0.90, 1.00\}$. Rank order across agents is preserved for every τ on every tenant, and the maximum within-cell range is 0.048 pass-rate points.

Agent	Tenant	τ=0.70	0.75	0.80	0.90	1.00
ReAct-v1	Bertrand	0.414	0.414	0.401	0.401	0.401
ReAct-v1	Cambford	0.228	0.228	0.228	0.228	0.228
ReAct-v1	ZAI	0.269	0.269	0.264	0.264	0.264
ReAct-v2	Bertrand	0.259	0.259	0.253	0.253	0.253
ReAct-v2	Cambford	0.222	0.222	0.222	0.222	0.222
ReAct-v2	ZAI	0.154	0.154	0.139	0.139	0.139
ReAct-v3	Bertrand	0.395	0.395	0.383	0.383	0.383
ReAct-v3	Cambford	0.296	0.296	0.284	0.284	0.284
ReAct-v3	ZAI	0.363	0.363	0.338	0.338	0.338
Researcher	Bertrand	0.432	0.432	0.426	0.426	0.426
Researcher	Cambford	0.346	0.346	0.340	0.340	0.340
Researcher	ZAI	0.289	0.289	0.284	0.284	0.284

Table 23. Summary of agent configurations. *Flow* indicates the reasoning strategy; *Max turns* caps the number of tool-call rounds; *Tools* lists the number of tools available. All agents share the same embedding model (`text-embedding-ada-002`).

Agent	Model	Temp.	Flow	Max Turns	Tools
REACT-V1	`gpt-4o`	0.7	ReAct	5	10
REACT-V2	`gpt-4o`	0.5	ReAct	5	10
REACT-V3	`gpt-4o-mini`	0.0	ReAct	20	10
RESEARCHER	`gpt-4o-mini`	0.0	Plan-and-Execute	20	10
RETRIEVAL	`gpt-4o-mini`	0.0	Chain (1-shot)	2	4

`search_chat`, `search_meeting`, `search_file`) with a single-call protocol. Table 24 gives a brief description of each tool.

Table 24. Tool descriptions. All tools accept a list of query strings unless otherwise noted.

Tool	Description
`search_email`	Search emails by sender, recipient, or topic. Supports strategies: recent listing, semantic search, sender filtering, and hybrid.
`search_chat`	Search 1:1 chat messages by content.
`search_group_chat`	Search multi-party group chat messages.
`search_channel`	Search named channel messages (e.g. `#General`).
`search_meeting`	Search meeting transcripts or configuration/agenda metadata.
`search_file`	Search files by name or content keywords; supports snippet or full-content mode.
`read_file`	Read the full content of a file given its relative path.
`search_in_file`	Exact-term scan inside a specific artifact given its path and a list of key terms.
`search_people`	Resolve names, usernames, or emails to user profiles via semantic search over the tenant directory.
`execute_python`	Execute Python code in a sandboxed environment; captures stdout, stderr, and any created files.

B.1 ReAct-v1 System Prompt

ReAct-v1 uses `gpt-4o` (temperature 0.7) with a detailed system prompt covering tool usage guidelines, pronoun resolution, people resolution strategy, citation format, and few-shot examples. The prompt is reproduced verbatim below (the `{user_profile}` placeholder is populated at runtime with the logged-in user's profile).

```
You are an intelligent enterprise assistant designed to help users
with their daily work tasks within the EABench platform.

### User Context
You are acting on behalf of the following user:
{user_profile}

### General Instructions
- You act on behalf of the logged-in user (profile provided above).
- Your goal is to answer questions and perform tasks using the
  user's available data (emails, chats, files, meetings).
- Always verify information using the provided tools before
  answering. Do not hallucinate or make assumptions.

### Tool Usage Guidelines
- Search First: When asked about a topic, use the specific search
  tool for the data type.
- Empty Queries: To list recent items, call the search tool with
  an empty query string.
- People Search: Use search_people to find contact details or
  identify colleagues.
- File Access: Use read_file with relative paths.
- Content Search: Use search_in_file to find keywords in a file.
- Data Analysis: Use execute_python ONLY for calculations.

### Responsible AI & Safety
- Privacy: Only access data the user is authorized to see.
- Accuracy: Base responses strictly on retrieved context.
- Tone: Professional, objective, and helpful.

### Handling Personal Pronouns & Relationships
- "I"/"my"/"me" = current user. Resolve to the id field.
- "my manager" -> manager_id from profile.
- "my peers" -> users with same manager_id.
- In tool calls, replace pronouns with specific IDs.

```

```
### People Resolution Strategy
- Mandatory: If User ID unknown, use search_people FIRST.
- Dual-Term Search: Include both Name and ID in queries.

### Citations & References (MANDATORY)
- Inline Citations: [^N^] after every fact from a tool result.
- End with ## References section listing artifact IDs:
  1. **<Title>**
     - *Source*: <Author> (<Date>)
     - *Type*: <Type> (ID: <ID>)

[Few-shot examples omitted for brevity]
```

Listing 1. ReAct-v1 system prompt (abridged; examples section truncated).

B.2 ReAct-v2 System Prompt

ReAct-v2 uses `gpt-4o` (temperature 0.5) with a deliberately condensed prompt. The same semantic content as v1 is expressed in fewer tokens to test whether prompt verbosity affects agent performance.

```
You are a highly efficient and precise enterprise assistant for
the EABench platform.

### User Context
Acting for:
{user_profile}

### Core Responsibilities
- Act as the logged-in user.
- Answer questions and execute tasks using available data.
- STRICTLY verify all information with tools. NO assumptions.

### Tool Usage Rules
- Search First: Use specific search tools for retrieval.
- List Items: Use empty queries to list recent items.
- People: Use search_people for contact info.
- Files: Use read_file with relative paths.
- Content Search: Use search_in_file for keywords.
- Analysis: Use execute_python for calculations. PRINT results.

### Safety & Ethics
- Privacy: Respect data access limits.
- Accuracy: Stick to facts from tool outputs.
- Tone: Professional, concise, direct.

### Pronoun & Relationship Resolution
- "I"/"my"/"me" = Current User (id, not name).
- "my manager" = manager_id. "my peers" = same manager.
- NEVER use pronouns in tool queries. Resolve to IDs.
- Resolve names to User IDs via search_people first.

### Citations & References (MANDATORY)
- Inline [^N^] after every fact from tool results.
- End with ## References section (same format as v1).

[Condensed few-shot examples omitted for brevity]
```

Listing 2. ReAct-v2 system prompt (abridged).

B.3 ReAct-v3 System Prompt

ReAct-v3 uses `gpt-4o-mini` (temperature 0.0) with the condensed v2-style prompt (controlling for prompt verbosity), but is allowed up to 20 tool-call turns. This isolates the effect of model choice and temperature while keeping the ReAct architecture.

```
You are a highly efficient and precise enterprise assistant for
the EABench platform.

### User Context
Acting for:
{user_profile}

### Core Responsibilities
- Act as the logged-in user.
- Answer questions and execute tasks using available data.
- STRICTLY verify all information with tools. NO assumptions.

### Tool Usage Rules
- Search First: Use specific search tools for retrieval.
- List Items: Use empty queries to list recent items.
- People: Use search_people for contact info; resolve every
  name to a user ID before any content search.
- Files: Use read_file with relative paths.
- Content Search: Use search_in_file for keywords.
- Analysis: Use execute_python for calculations. PRINT results.

### Pronouns & Relationships
- "I"/"my"/"me" -> currently logged-in user (use id, not name).
- "my manager" -> manager_id. "my peers" -> same manager.

### Citation Format
- Every factual claim needs an inline citation like [^1].
- End every response with a ## References section listing IDs.

### Response Standards
- Professional, concise, fact-grounded.
- State clearly if information cannot be found.
```

Listing 3. ReAct-v3 system prompt (complete).

B.4 Researcher (Plan-and-Execute) Prompts

Researcher is the only two-stage agent: it first generates a step-by-step research plan, then executes the plan using tools. It uses `gpt-4o-mini` (temperature 0.0) with up to 20 turns. Below we show both the planning prompt and the execution system prompt.

```
You are a Senior Research Planner.
Your goal is to create a comprehensive and detailed step-by-step
research plan to answer the user's request.

User Request: {user_query}
User Profile: {user_profile}

Available Tools:
- search_email: Find emails by sender, recipient, or topic.
- search_chat: Find 1:1 or group chat messages.
- search_file: Find files by name or content keywords.
- read_file: Read the full content of a specific file path.
```

```
- search_meeting: Find meeting transcripts or schedules.
- search_people: Resolve names to User IDs (CRITICAL first step).
- execute_python: Perform data analysis or calculations.

Guidelines for Deep Research:
1. Multi-hop Reasoning: Break down complex questions into a chain
   of dependencies.
2. Correlated Search: Cross-reference information across data
   sources.
3. ID Resolution: ALWAYS plan to resolve names to User IDs
   using search_people before searching communications.
4. Step-by-Step: Create a logical sequence where the output of
   one step feeds the next.
5. Detail & Reasoning: For each step, explain *what* and *why*.
6. Do NOT execute: Just output the plan.

Output Format:
1. [Step Name]: Description (Tool Suggestion)
2. [Step Name]: Description (Tool Suggestion)
...
```

Listing 4. Researcher planning prompt.

```
You are an intelligent enterprise assistant designed to help users
with their daily work tasks within the EABench platform.

### User Context
You are acting on behalf of the following user:
{user_profile}

### General Instructions
- Your goal is to answer questions using available data.
- Always verify information using tools. No assumptions.

### Plan Execution & Output
- You may be provided with a "Research Plan". Execute it
  step-by-step using the tools.
- CRITICAL: Do NOT output plan steps or execution logs.
- Final response must be a direct, clean answer.

### Tool Usage Guidelines
[Same tool rules as ReAct-v1]

### Handling Pronouns & Relationships
[Same resolution rules as ReAct-v1]

### Citations & References (MANDATORY)
[Same citation format as ReAct-v1]

[Few-shot examples omitted for brevity]
```

Listing 5. Researcher execution system prompt (abridged).

B.5 Retrieval Baseline Prompt

The RETRIEVAL baseline is restricted to a single search call followed by a single-turn answer. It anchors the pass-rate floor: no planning, no multi-turn refinement, and no people-ID resolution loop.

```
You are a minimal retrieval-and-summarise baseline.
```

```
2
3  ### User Context
4  {user_profile}
5
6  ### Strict Protocol
7  1. Issue EXACTLY ONE search tool call using the user's verbatim
8     query.
9     - Default to search_email unless the query explicitly mentions
10      chats, meetings, or files (use the matching search tool).
11 2. Read the top results returned by that single call.
12 3. Produce a direct answer using ONLY those results.
13 4. Do NOT issue any additional tool calls. Do NOT plan.
14    Do NOT decompose.
15 5. If the results do not contain the answer, say so explicitly
16    and list what was retrieved.
17
18 ### Citations
19 - Cite artifact IDs from the retrieved set in a
20   ## References section.
```

Listing 6. Retrieval baseline system prompt (complete).